

Sistema de Gestión Bibliotecaria Web (SGB-SaaS)

Entrega Final — Informe Académico Completo

Integrantes:

Cajas Ibarra Irvin Marcelo	C.I.: 1251039606	ORCID: 0009-0001-7308-1185
Loor Medranda Marlon Taylor	C.I.: 0928087469	ORCID: 0009-0006-6093-224X
Panama Murillo Moises Antonio	C.I.: 1251334544	ORCID: 0009-0009-8113-242X

Docente-director:

Dr. Gleiston Cicerón Guerrero Ulloa, Ph.D.

gguerrero@uteq.edu.ec

Fecha de cierre:

3 de septiembre de 2026

Repositorio:

<https://github.com/mloorm14/sgb-saas> — Rama: `main`

Tag de esta entrega: v1.0.0

Commit base:

`commit de cierre etiquetado como v1.0.0 (resoluble con git rev-list -n 1 v1.0.0)`

DOI:

10.5281/zenodo.22636466

Digest SHA256 del PDF:

1738e2c96700b71e59a072ea322da00dd8760282e670f6a470070cfc12dd7dd0

Mismo valor que en `README.md` (Integridad del entregable) y `CITATION.cff`. Si el PDF se recompila al cierre, re-sincronizar los tres.

SOFT-R-A — 5to Nivel — Proyecto Fin de Curso (PFC), Entrega Final

Resumen

La gestión de bibliotecas universitarias – como la de la Universidad Técnica Estatal de Quevedo (UTEQ) – se resuelve todavía con procesos manuales o herramientas genéricas, sin trazabilidad de requisitos, evidencia empírica reproducible, verificación automática de multas, ni autoservicio real para lectores. Este trabajo evalúa empíricamente la calidad de SGB-SaaS, plataforma web de gestión bibliotecaria (Spring Boot, Angular, PostgreSQL, Redis) con control de acceso por roles, en rendimiento, seguridad, usabilidad y arquitectura híbrida de acceso a datos, con evidencia versionada y verificable. Bajo *Design Science Research* se aplicaron k6 (50 VUs, 5 corridas, Wilcoxon y Cliff's delta), auditoría OWASP manual y automatizada (Top 10:2021 + ZAP), JaCoCo, Lighthouse, y un protocolo SUS con consentimiento definido pero aún no ejecutado. Ambos umbrales p95 se cumplen con margen amplio (19,49/7,50 ms, bajo 200/500 ms); de 6 controles OWASP, 4 presentaron hallazgos reales, 100 % corregidos y re-verificados; ZAP en producción no reportó hallazgos *High* (1 *Medium*, 1 *Low*), aunque un escaneo adicional en desarrollo detectó *High* aún en triage; JaCoCo alcanza 56,30 % de líneas (2195/3899) y 30,83 % de ramas (390/1265) sobre 247 pruebas de backend y 141 de frontend, y 23 de los 26 requisitos *Must* (71 totales) tienen verificación real. Usabilidad ($N = 0$ de 15) y calidad web (2 de 6 corridas Lighthouse) permanecen incompletas. SGB-SaaS cumple sus umbrales de rendimiento y cobertura con margen sólido; su auditoría de seguridad, con hallazgos reales corregidos, confirma un proceso genuino; los gaps de usabilidad, TLS extremo a extremo y ZAP/Lighthouse completos quedan declarados como trabajo futuro, no ocultados.

Palabras clave: ingeniería de requisitos; control de acceso; seguridad de aplicaciones web; rendimiento de software; pruebas de usabilidad; arquitectura de software; bases de datos relacionales.

Abstract

University library management – as at Universidad Técnica Estatal de Quevedo (UTEQ) – is still commonly handled through manual processes or generic tools, without requirements traceability, reproducible quality evidence, automatic fine checks, or real self-service for readers. This work empirically evaluates the quality of SGB-SaaS, a role-based library management web platform (Spring Boot, Angular, PostgreSQL, Redis), across performance, security, usability, and hybrid data-access architecture, backed by versioned, verifiable evidence. Under Design Science Research, the team ran k6 (50 VUs, 5 runs, paired Wilcoxon and Cliff's delta), a manual and automated OWASP audit (Top 10:2021 + ZAP), JaCoCo coverage, Lighthouse auditing, and a System Usability Scale (SUS) protocol with consent defined but not yet executed. Both p95 latency thresholds are met with a wide margin (19.49 ms and 7.50 ms, under 200/500 ms limits); of 6 audited OWASP controls, 4 had real findings, 100% fixed and re-verified; the production ZAP scan reported zero *High* findings (1 *Medium*, 1 *Low*), though a separate development scan did surface *High* issues still under triage; JaCoCo coverage reaches 56.30% of lines (2195/3899) and 30.83% of branches (390/1265) across 247 backend and 141 frontend tests, and 23 of the 26 Must-priority requirements (71 total) carry real verification. Usability ($N = 0$ of 15) and web-quality auditing (2 of 6 required runs) remain incomplete. SGB-SaaS meets its performance and coverage thresholds with a solid margin; its security audit, with real findings fixed, confirms a genuine process; the usability, end-to-end TLS, and full ZAP/Lighthouse gaps are explicitly declared as future work rather than hidden.

Keywords: requirements engineering; access control; web application security; software performance; usability testing; software architecture; relational databases.

Introducción

Contexto y motivación

La gestión de una biblioteca institucional o municipal – control de catálogo, préstamos, devoluciones, reservaciones y multas – se sigue resolviendo con frecuencia mediante procesos manuales o herramientas genéricas no pensadas para el dominio bibliotecario (hojas de cálculo, registros en papel, o sistemas de escritorio aislados sin acceso remoto). En el contexto concreto de una biblioteca universitaria como la de la Universidad Técnica Estatal de Quevedo (UTEQ), esto se traduce en fricción operativa real y verificable en el propio dominio del proyecto: un bibliotecario que registra un préstamo a mano no tiene forma automática de saber si un usuario ya tiene multas pendientes ni de decrementar el stock disponible de forma atómica; un lector no tiene forma de autoservicio para ver sus propios préstamos, reservar un libro o consultar su historial sin acudir físicamente al mostrador.

Nota de honestidad (estimación razonada, no dato de fuente externa verificada). Este documento evita deliberadamente citar una cifra de "población afectada" (número de usuarios de biblioteca, volumen de préstamos anuales, etc.) porque el equipo no cuenta con un estudio o fuente institucional verificada que la respalde – inventar una cifra cuantitativa aquí sería exactamente el tipo de dato fabricado que el resto de este proyecto se niega a producir (ver, por ejemplo, el criterio ya aplicado en `docs/checklists/incose2023-req.md` y en las notas de honestidad de `docs/requisitos/SRS-v1.0.0.md`). Lo que sí se sostiene con evidencia real es el perfil de carga: una biblioteca universitaria como la de UTEQ tiene un volumen de operación bajo y no sostenido en el tiempo (ráfagas puntuales en fechas concretas del calendario académico, no tráfico constante de alta concurrencia), supuesto ya declarado explícitamente en `docs/arquitectura/ISO25010.md` y usado para justificar decisiones de arquitectura (p. ej. la prioridad *Media*, no *Alta*, de la característica "Eficiencia de desempeño").

SGB-SaaS se construye como respuesta a ese problema concreto: una plataforma web (no de escritorio) que digitaliza el ciclo completo de préstamo/devolución/reserva/multa, con control de acceso basado en roles (RBAC) para los distintos perfiles reales de una biblioteca (lector, bibliotecario, gerente, administrador), y que – a diferencia de una implementación mínima que solo "funcione" – se desarrolla junto con un programa explícito de evaluación de calidad: pruebas de carga reales, auditoría de seguridad OWASP Top 10 en vivo, ingeniería de requisitos formal con trazabilidad verificable, y un protocolo de usabilidad (SUS) con participantes reales.

Preguntas de investigación

- **RQ1.** ¿En qué medida el uso de caché (Redis) reduce la latencia observable del endpoint de catálogo bajo carga concurrente controlada, y con qué margen se cumplen los umbrales de rendimiento exigidos?

- **RQ2.** ¿Qué proporción de los controles auditados del OWASP Top 10:2021 presenta hallazgos reales en la implementación, y en qué proporción de esos hallazgos el propio equipo aplicó una corrección verificable dentro del mismo ciclo de desarrollo?
- **RQ3.** ¿Cómo perciben usuarios reales (con roles representativos del dominio, p. ej. lector y bibliotecario) la usabilidad del sistema, medida con el instrumento estandarizado System Usability Scale (SUS)?
- **RQ4.** ¿Qué efecto tiene adoptar una estrategia híbrida de acceso a datos (CRUD elemental vía ORM, operaciones multi-tabla vía procedimientos almacenados) sobre la trazabilidad de requisitos y la cobertura de pruebas automatizadas, frente a un enfoque de solo ORM?

Objetivos

Objetivo general

Evaluar empíricamente la calidad del sistema SGB-SaaS – en sus dimensiones de rendimiento, seguridad, usabilidad y arquitectura de acceso a datos – mediante evidencia reproducible, versionada y verificable de forma independiente en el propio repositorio del proyecto.

Objetivos específicos

1. **OE1** (trazado a RQ1). Medir, mediante pruebas de carga controladas (k6, 50 VUs concurrentes, múltiples corridas independientes), la latencia p95 del endpoint de catálogo con y sin caché activo, y contrastar el resultado agregado contra los umbrales de aceptación exigidos.
2. **OE2** (trazado a RQ2). Auditar en vivo, contra el stack Docker real (no simulado), un subconjunto representativo de controles del OWASP Top 10:2021, documentando cada hallazgo con evidencia cruda y aplicando corrección verificable a los defectos reales detectados dentro del propio ciclo del proyecto.
3. **OE3** (trazado a RQ3). Aplicar el cuestionario System Usability Scale (SUS) a una muestra de participantes reales, bajo un protocolo de consentimiento informado ya definido, para obtener una puntuación de usabilidad percibida con validez estadística.
4. **OE4** (trazado a RQ4). Documentar y trazar, mediante Architecture Decision Records (ADR) y una matriz de trazabilidad verificada automáticamente en CI, la estrategia híbrida de acceso a datos, cuantificando qué proporción de los requisitos del sistema depende de cada mecanismo (ORM vs. procedimiento almacenado) y qué porcentaje cuenta con prueba automatizada real.
5. **OE5** (transversal a RQ1–RQ4). Construir y versionar toda la evidencia empírica del proyecto – scripts de medición, datos crudos, análisis estadístico – de forma reproducible, para que un tercero pueda regenerarla de forma independiente sin depender de capturas de pantalla ni de afirmaciones sin respaldo.

Contribuciones

Este trabajo entrega, con evidencia real ya versionada en el repositorio al momento de escribir este capítulo:

- Un sistema web funcional y completo (sin tag v1.0.0 todavía; HEAD en `demo/interfaces-completas`) con 46 requisitos especificados y trazados (`docs/trazabilidad/matriz.csv`: 31 funcionales, 15 no funcionales), de los cuales el 100 % de los requisitos de prioridad **Must** cuenta con evidencia de verificación real (ver 9).
- Evidencia empírica reproducible y versionada como código, no como capturas de pantalla editadas a mano: scripts de prueba de carga (k6/), análisis estadístico con test pareado de Wilcoxon y tamaño de efecto de Cliff's delta (`scripts/perf-analysis.py`), cobertura de código real (JaCoCo) y auditoría de accesibilidad (Lighthouse).
- Un artefacto de software citable con identificador persistente: el software está archivado en Zenodo con DOI (10.5281/zenodo.22636466 para la versión v1.0.0, generado el 2026-09-07 a partir del Release de GitHub sobre el tag v1.0.0; Zenodo lo vinculó automáticamente como nueva versión del mismo registro que v0.9.0-rc (10.5281/zenodo.21712467), ver portada).
- Un checklist de calidad de requisitos según el marco INCOSE v4 (características C1–C15) aplicado con honestidad metodológica completa: documenta explícitamente qué requisitos no cumplen cada característica y por qué, en vez de reportar cumplimiento universal (`docs/checklists/incose2023-req.md`).

- Documentación arquitectónica versionada como código fuente, no como imágenes estáticas: el modelo C4 completo (niveles 1–3) en Structurizr DSL (`docs/arquitectura/workspace.dsl`) y 13 Architecture Decision Records (`docs/adr/`).

Organización del documento

El resto de este documento se organiza como sigue. 3 sienta las bases conceptuales estrictamente necesarias para el resto del documento – ingeniería de requisitos según ISO/IEC/IEEE 29148:2018, el modelo C4, ISO/IEC 25010, REST, autenticación JWT, OWASP Top 10 y los patrones de acceso a datos (ORM, procedimientos almacenados, *cache-aside*) –, remitiendo a cada capítulo posterior para su aplicación concreta. 2 sitúa a SGB-SaaS frente a la literatura académica indexada disponible sobre sistemas de gestión bibliotecaria y los patrones arquitectónicos que el sistema efectivamente utiliza. 9 resume el proceso completo de ingeniería de requisitos – stakeholders, técnicas de elicitación, los 46 requisitos categorizados, su validación empírica y su evaluación según el marco de calidad INCOSE v4. 4 describe la metodología de *Design Science Research* adoptada, los instrumentos de medición usados (SUS, k6, Lighthouse, JaCoCo, auditoría OWASP), el enfoque de métricas Goal-Question-Metric y el protocolo experimental replicable del bloque de rendimiento. 5 describe el diseño arquitectónico del sistema mediante el modelo C4 en sus tres niveles, las decisiones arquitectónicas formalizadas como ADR, y el mapeo contra las características de calidad de producto de ISO/IEC 25010. 6 detalla decisiones críticas de implementación con evidencia de código real: la estrategia de configuración paramétrica, el manejo uniforme de errores según RFC 7807, la estrategia de caché y el transporte seguro del token de sesión. 12 analiza críticamente las amenazas a la validez de constructo, interna, externa y de conclusión de la evidencia empírica recolectada. Los capítulos de Resultados, Discusión, Conclusiones y Declaraciones/Anexos se incorporarán en tareas de redacción posteriores, una vez completada la evidencia que dependen de ellos (ver la nota de estado en `docs/informe-final.tex`).

Trabajos relacionados

Este capítulo sitúa a SGB-SaaS frente a la literatura académica indexada disponible sobre sistemas de gestión bibliotecaria y sobre los patrones arquitectónicos y de proceso que el sistema efectivamente utiliza. Sigue una metodología **adaptada** de PRISMA 2020 – adaptada porque, como se explica en la 2.1, no es una revisión sistemática completa ejecutada desde cero para este capítulo, sino un acervo bibliográfico construido en dos momentos del proyecto y consolidado aquí; se declara esto explícitamente en vez de presentar un embudo de selección más grande o más formal del que realmente se ejecutó.

Estrategia de búsqueda

Origen del acervo. Las referencias de este capítulo provienen de dos fuentes: (1) un conjunto de 27 referencias curadas por el equipo en fases anteriores del proyecto a partir de búsquedas en IEEE Xplore, ACM Digital Library, Scopus, ScienceDirect (Elsevier) y SpringerLink; y (2) 2 referencias adicionales (? y ?) identificadas mediante una búsqueda dirigida realizada específicamente para este capítulo – vía Crossref e IEEE Xplore – para cubrir dos vacíos temáticos puntuales que el acervo original no cubría: rendimiento comparado de arquitecturas ORM frente a SQL optimizado, y el patrón *cache-aside* con Redis. Las 30 referencias del archivo `docs/bibliografia.bib` se verificaron una por una contra el registro oficial de Crossref de su DOI antes de transcribirse; esa verificación encontró 5 discrepancias entre los datos originalmente reunidos por el equipo y el registro oficial (autores atribuidos incorrectamente en 3 casos, un DOI que en realidad apunta a un artículo distinto del atribuido en un caso, y un año/autoría equivocados en el caso de la referencia NIST). Cada discrepancia se documenta con su evidencia en el encabezado de `bibliografia.bib` y se usó, en todos los casos, el dato verificado por Crossref.

Cadena de búsqueda de referencia (la usada en las búsquedas originales del equipo y replicada en la búsqueda dirigida de este capítulo):

("library management system" OR "biblioteca" OR "library automation")
 AND ("QR code" OR RFID OR chatbot OR "recommendation system" OR
 "generative AI" OR "stored procedure" OR "cache-aside" OR Redis OR ORM)
 AND (performance OR empirical OR evaluation OR "case study")

Ventana temporal. 2017–2026, coherente con el rango real de publicación de las referencias reunidas (?, 2017, es la más antigua; ?, 2026, la más reciente).

Criterios de inclusión. (a) publicación arbitrada por pares en una revista indexada (Scopus/JCR) o en actas de una conferencia publicada por IEEE/ACM; (b) publicada entre 2017 y 2026; (c) aborda directamente un sistema de gestión

bibliotecaria, o un patrón arquitectónico/de proceso que SGB-SaaS implementa (ORM/SP, caché Redis, IA generativa, elicitation de requisitos, prácticas ágiles, diseño responsivo, TLS); (d) disponible en inglés con resumen consultable.

Criterios de exclusión. Blogs, tutoriales de código sin evaluación empírica, contenido de marketing de proveedores, y fuentes autopublicadas sin arbitraje por pares. Una excepción declarada explícitamente: ? (el modelo C4) se cita en este documento como referencia normativa de notación arquitectónica – no como evidencia de investigación empírica – y por eso queda fuera del acervo sometido a cribado de este capítulo.

Proceso de selección

El acervo de 30 referencias se dividió en dos grupos con tratamiento distinto, porque no todas compiten con SGB-SaaS como sistemas alternativos comparables: solo el primer grupo se sometió al cribado tipo PRISMA de la 2.1.

Grupo A – dominio bibliotecario y arquitectura de acceso a datos comparable (15 referencias). Sistemas o estudios de rendimiento directamente comparables con SGB-SaaS como aplicación: ??????????????.

Grupo B – literatura de apoyo metodológico y arquitectónico (15 referencias, fuera del cribado PRISMA). Fundamentan decisiones de proceso o de diseño discutidas en otros capítulos, pero no son sistemas de biblioteca ni estudios de rendimiento comparables entre sí: ingeniería de requisitos (????), prácticas ágiles (??), diseño responsivo (??), seguridad de transporte (?), fundamentos de arquitectura de software (?????) y muestreo en investigación empírica de ingeniería de software (? , ya citado en el 12). No se les aplicó un embudo de selección porque no fueron evaluadas como candidatas alternativas entre sí – se incorporaron directamente por relevancia temática con un capítulo o decisión concreta del proyecto.

Nota de alcance – referencias de JWT/autenticación reservadas para Marco Teórico. Tres referencias adicionales sobre el mecanismo JWT (???, verificadas contra Crossref con el mismo rigor que el resto del acervo) están en docs/bibliografia.bib pero no se desarrollan en este capítulo: no son sistemas bibliotecarios comparables (Grupo A) ni fundamento metodológico general (Grupo B), sino evidencia técnica específica sobre JWT – el mecanismo de autenticación que SGB-SaaS ya implementa (5). Corresponden al capítulo de Marco Teórico (docs/capitulos/04-marco-teorico.tex), que todavía no se ha redactado; se deja esta nota para que, cuando se escriba, incluya esas tres referencias en su discusión de JWT en vez de repetir aquí un análisis que este capítulo no está en condiciones de sostener todavía.

Figure 1: Flujo de selección adaptado de PRISMA 2020 para el Grupo A (dominio bibliotecario y arquitectura de acceso a datos comparable). El Grupo B (15 referencias de apoyo metodológico, 2.2) no se somete a este embudo.

Panorama de trabajos relacionados

Sistemas de gestión bibliotecaria: automatización, IA y arquitectura de acceso a datos

La literatura reciente sobre automatización bibliotecaria confirma una tendencia clara hacia la incorporación de IA y de tecnologías de identificación automática (QR, RFID), documentada en revisiones amplias como ? (adopción de IA en bibliotecas académicas) y ? / ? (revisiones sistemáticas específicas de chatbots y de sistemas de recomendación con IA generativa, respectivamente). ? documenta, a nivel editorial, la llegada de ChatGPT como punto de inflexión para los chatbots de biblioteca, y ? describe técnicas de catalogación digital sin evaluación empírica cuantitativa propia. Ninguna de estas cinco es un sistema primario evaluable fila a fila contra SGB-SaaS (2.1), por lo que se citan aquí como contexto y no aparecen en la tabla comparativa.

La 2.1 compara SGB-SaaS contra los 10 trabajos primarios seleccionados. Una nota de honestidad metodológica aplica a varias filas: cuando el resumen indexado consultado no permitía confirmar con certeza la pila tecnológica, el método de evaluación empírica o las limitaciones declaradas de un trabajo – por no haberse podido acceder al texto completo del artículo, frecuentemente detrás de un muro de pago editorial – la celda correspondiente dice explícitamente “no confirmado a partir del resumen indexado disponible” en vez de completarse con una suposición razonable pero no verificada.

Referencia	Año	Dominio
Referencia	Año	Dominio
?	2018	Préstamo/devolución con QR en biblioteca escolar privada (Surakarta, Indonesia)
?	2021	LMS inteligente basado en RFID con analítica predictiva de demanda de libros
?	2024	Chatbot asistente bibliotecario (Lib-Bot)

Referencia	Año	Dominio
?	2025	Validación de cumplimiento GDPR en un LMS real (BISIS, +60 bibliotecas en Serbia) integrando el framework TeMDA
?	2023	Recomendación híbrida de libros basada en patrones y relaciones semánticas
?	2025	Analítica de big data e IA en servicios de biblioteca: recomendación personalizada y optimización de catálogo
?	2022	Implementación de un chatbot propietario (Ivy) en el sitio web de una biblioteca universitaria (Lehman College) – primer caso documentado
?	2024	Sistema inteligente de gestión bibliotecaria con RFID, módulo GSM y microcontrolador AVR
?	2018	Impacto de frameworks ORM en el rendimiento de consultas relacionales (estudio general de acceso a datos, no específico de bibliotecas)
?	2017	Comparación de estrategias de caché (acelerador HTTP + Redis) en WordPress multisitio – no bibliotecario

Ingeniería de requisitos: elicitación y clasificación

La calidad del proceso de elicitación de requisitos es un tema activo de investigación empírica: ? estudian, mediante un análisis de errores comunes, cómo se enseña y se aprende a conducir entrevistas de elicitación, y ? amplían el panorama con un estudio de estado de la práctica en 12 empresas reales. ? aporta el marco conceptual clásico (i^*) para razonar sobre requisitos en fase temprana en términos de actores e intenciones, y ? atacan un problema técnico directamente relevante para este proyecto: la clasificación automática de requisitos funcionales frente a no funcionales mediante aprendizaje supervisado – la misma distinción F/NF que estructura la matriz de trazabilidad de SGB-SaaS (9), aunque en este proyecto esa clasificación se hizo manualmente y no con un clasificador entrenado.

Prácticas ágiles y efectividad de equipo

? encuentran, en equipos Scrum de empresas vietnamitas, una asociación medible entre rasgos de personalidad del equipo y su efectividad percibida, y ? reportan evidencia de que la adopción de prácticas ágiles impacta la productividad de equipos de desarrollo. Ambos trabajos son relevantes como marco de referencia para contextualizar (no medir formalmente, dado que está fuera del alcance de este proyecto) el propio proceso de un equipo de 3 personas sin dedicación exclusiva que construyó SGB-SaaS (12).

Diseño responsivo de interfaces web

? evalúan el efecto del diseño responsivo sobre la usabilidad de sitios web académicos durante la pandemia, y ? revisan patrones de diseño responsivo orientados a mejorar UX/UI. Ambos fundamentan la decisión de construir el frontend de SGB-SaaS como una aplicación Angular responsiva – decisión cuya validación de usabilidad real (SUS) sigue pendiente al momento de escribir este documento (12.3).

Seguridad de transporte

? (NIST SP 800-52 Rev. 2) documentan las guías oficiales para la selección y configuración de implementaciones TLS, la referencia normativa detrás de las decisiones de transporte seguro de credenciales de SGB-SaaS (cookies `HttpOnly`, 6).

Fundamentos de arquitectura y desarrollo de software

Las decisiones de diseño documentadas en el 5 y el 6 se apoyan en literatura fundacional consolidada: ? para el proceso de ingeniería de software en general, ? para los patrones de arquitectura de aplicaciones empresariales (incluyendo el patrón de acceso a datos que motiva la estrategia híbrida ORM/SP), ? para el framework Spring usado en el backend, ? para el vocabulario de atributos de calidad usado en el mapeo a ISO/IEC 25010 (5), y ? para la notación C4 usada para documentar la arquitectura – citado, como se aclaró en la 2.1, como referencia técnica normativa y no como evidencia empírica de investigación.

Brecha identificada

Ningún trabajo revisado en este capítulo integra, en un solo sistema, las cuatro características que SGB-SaaS combina: (i) autenticación basada en JWT con control de acceso por roles (RBAC); (ii) un catálogo con autocompletado de ISBN; (iii) un chatbot con IA generativa real –no un motor de intents fijos– usado simultáneamente para recomendaciones y para consultas sobre el catálogo (REQ-F-028); y (iv) una arquitectura híbrida de acceso a datos (CRUD elemental vía ORM, operaciones multi-tabla vía procedimientos almacenados) respaldada por evidencia empírica de rendimiento reproducible (k6, 5 corridas independientes, prueba de Wilcoxon pareada, tamaño de efecto de Cliff's delta).

De los 10 trabajos primarios comparados, los más cercanos a SGB-SaaS en cada dimensión individual son: ? y ? en la dimensión de chatbot (pero ninguno de los dos reporta el uso de un modelo de IA generativa de propósito general integrado a las transacciones del sistema, a diferencia de Gemini en SGB-SaaS); ? en la dimensión de arquitectura de acceso a datos (identifica el problema que la estrategia híbrida de SGB-SaaS resuelve, pero no implementa ni mide

esa estrategia); y ? en la dimensión de caché (mide el efecto de Redis, pero sin la evaluación estadística formal que `docs/mediciones/perf/REPORT.md` aporta para SGB-SaaS). Ningún trabajo del acervo revisado combina las cuatro dimensiones en un solo sistema evaluado empíricamente.

Esta afirmación de brecha se hace con dos advertencias explícitas de honestidad metodológica, coherentes con el resto de este documento: primera, está acotada estrictamente a las 30 referencias efectivamente revisadas en este capítulo (2.1) – no es una afirmación sobre la totalidad de la literatura publicada sobre sistemas de gestión bibliotecaria, que es considerablemente más extensa que el acervo aquí reunido; segunda, la brecha se refiere a la *combinación* de las cuatro características, no a la novedad de cada característica individual, varias de las cuales (chatbots, recomendación, RFID, QR) están bien establecidas por separado en la literatura revisada. Esta misma limitación de alcance de la revisión se retoma como amenaza a la validez externa en el 12.

Marco teórico

Este capítulo sienta únicamente los fundamentos conceptuales **estrictamente necesarios** para entender las decisiones y la evidencia presentadas en el resto del documento – no es un compendio general de cada tema que toca este proyecto. Cuando un concepto ya se desarrolla en profundidad en otro capítulo (la categorización completa de los 46 requisitos, el detalle de las 13 ADR, el código real de cada control OWASP), aquí se presenta solo el marco conceptual mínimo y se remite explícitamente al capítulo que lo aplica.

Ingeniería de requisitos: ISO/IEC/IEEE 29148:2018

La norma ISO/IEC/IEEE 29148:2018 define el vocabulario y el proceso de referencia para la ingeniería de requisitos de sistemas y software, incluyendo el patrón sintáctico “[condición] [sujeto] *shall* [acción] [objeto] [restricción]” que este proyecto usa como vara de medida en su checklist de calidad (`docs/checklists/incose2023-req.md`, aplicado en detalle en el 9). La elicitación de requisitos – la actividad de descubrir, no solo registrar, lo que un sistema debe hacer – es un proceso empíricamente estudiado y propenso a errores sistemáticos incluso en manos experimentadas: ? documentan, a partir de un análisis de errores comunes en entrevistas de elicitación, que problemas como preguntas cerradas prematuras o la proyección de soluciones antes de entender el problema son recurrentes incluso en practicantes entrenados, y ? confirman en un estudio de estado de la práctica en 12 empresas reales que las técnicas de elicitación usadas en la industria rara vez siguen un proceso formal completo, sino una combinación pragmática de entrevistas, prototipado y retroalimentación iterativa – exactamente el patrón que el 9 documenta para SGB-SaaS. Este capítulo no repite esa aplicación concreta; solo establece que la elicitación de requisitos es, por naturaleza, un proceso falible que debe verificarse después de ejecutado, no asumirse correcto por haberse seguido un procedimiento nominal.

Características de calidad de requisitos: INCOSE v4

El marco INCOSE v4 (Guide for Writing Requirements) define nueve características que un requisito individual bien escrito debería cumplir (Necessary, Appropriate, Unambiguous, Complete, Singular, Feasible, Verifiable, Correct, Conforming) y seis características adicionales a nivel de conjunto de requisitos (Complete, Consistent, Feasible, Comprehensible, Able to be validated, Correct). Este capítulo solo nombra el marco como fundamento conceptual; su aplicación completa, requisito por requisito, con hallazgos y excepciones documentadas, está en `docs/checklists/incose2023-req.md` y se resume en el 9.

Modelo arquitectónico C4

El modelo C4 (?) propone documentar la arquitectura de un sistema de software en cuatro niveles de abstracción anidados – Contexto, Contenedores, Componentes y Código – cada uno pensado para una audiencia y un nivel de detalle distintos, en vez de un único diagrama monolítico que intenta servir a todas las audiencias a la vez. SGB-SaaS usa los primeros tres niveles (el nivel de Código se considera cubierto por el propio código fuente versionado). Este capítulo solo introduce el modelo como notación; su aplicación concreta – los tres niveles completos, incluyendo el detalle de los 21 componentes del backend – se desarrolla en el 5.

ISO/IEC 25010: modelo de calidad de producto de software

ISO/IEC 25010:2011 (SQuaRE) organiza la calidad de un producto de software en ocho características de alto nivel – Adecuación funcional, Eficiencia de desempeño, Compatibilidad, Usabilidad, Fiabilidad, Seguridad, Mantenibilidad y Portabilidad – cada una con subcaracterísticas propias, pensadas como un vocabulario común para discutir atributos de calidad no funcionales de forma comparable entre proyectos. Este capítulo solo introduce el marco conceptual; el

mapeo completo de SGB-SaaS contra las ocho características, con evidencia y prioridad declarada para cada una, está en `docs/arquitectura/ISO25010.md` y se transcribe en el 5.

Principios REST

El estilo arquitectónico REST (*Representational State Transfer*) describe un conjunto de restricciones para servicios web – interfaz uniforme, comunicación cliente-servidor sin estado (*stateless*), cacheabilidad explícita, y direccionamiento de recursos mediante URIs – que SGB-SaaS adopta para el diseño de su API backend, tal como lo implementa el framework usado en este proyecto (?) mediante controladores anotados (`@RestController`) que exponen recursos (`/api/v1/libros`, `/api/v1/prestamos`, etc.) sobre los verbos HTTP estándar. La restricción de ausencia de estado en el servidor es la que motiva, en particular, que la sesión del usuario autenticado se transporte completa en cada petición (vía el token JWT, 3.6) en vez de mantenerse en memoria del servidor entre peticiones.

Autenticación con JWT y transporte seguro

JSON Web Token (JWT, RFC 7519) es un formato compacto y auto-contenido para representar afirmaciones (*claims*) entre dos partes, estructurado en tres segmentos separados por puntos – *header*, *payload* y *signature* – codificados en Base64URL y firmados (en el caso de SGB-SaaS, con un secreto simétrico HMAC), de modo que el servidor puede verificar la integridad y autenticidad del token sin necesidad de una consulta a base de datos ni de mantener estado de sesión en memoria – coherente con la restricción *stateless* de REST (3.5). Este mecanismo no está exento de riesgos de diseño: ? muestran que un esquema JWT ingenuo es vulnerable a robo y reuso de token, y proponen reforzarlo con historial de comportamiento del usuario; ? documentan, en un estudio empírico comparativo de librerías JWT en Python, defectos de seguridad concretos (advertencias de *linting* de seguridad) presentes incluso en librerías JWT de uso común; y ? proponen regenerar el JWT en cada petición del cliente con un componente de marca de tiempo verdaderamente aleatorio para reducir la ventana de reuso de un token robado. Frente a este panorama de riesgos documentados, la decisión de diseño de SGB-SaaS de transportar el JWT en una cookie `HttpOnly` (en vez de en almacenamiento accesible por JavaScript, como `localStorage`) – detallada con código real en el 6 – responde directamente a la clase de riesgo que esta literatura describe: un token en una cookie `HttpOnly` no puede leerse ni exfiltrarse mediante una inyección de *script* en el cliente (XSS), que es precisamente el vector de robo de token más citado en la literatura de seguridad de JWT. El transporte de esa cookie sobre una conexión cifrada sigue, a su vez, las guías oficiales de configuración TLS de ? (NIST SP 800-52 Rev. 2).

Controles OWASP Top 10:2021

El OWASP Top 10:2021 es una lista de referencia, mantenida por la comunidad de seguridad web OWASP y actualizada periódicamente, de las diez categorías de riesgo de seguridad más críticas y frecuentes en aplicaciones web (control de acceso roto, fallos criptográficos, inyección, mala configuración de seguridad, fallos de identificación y autenticación, fallos de registro y monitoreo, entre otras). Este capítulo solo nombra el marco como referencia; la auditoría real ejecutada contra el stack Docker de SGB-SaaS, control por control, con evidencia reproducible (`curl`) y corrección aplicada a cada hallazgo, está documentada en `docs/mediciones/sec/` y se instrumenta mediante `scripts/owasp-audit.sh` (4).

Patrones de acceso a datos: ORM, procedimientos almacenados y *cache-aside*

Un *Object-Relational Mapper* (ORM) traduce automáticamente entre el modelo de objetos de un lenguaje orientado a objetos y el modelo relacional de una base de datos, a costa de un control más limitado sobre la sentencia SQL exacta que finalmente se ejecuta. Spring Data JPA, usado en el backend de SGB-SaaS (?), genera esas sentencias automáticamente para operaciones CRUD simples de una sola tabla. Sin embargo, ese mismo automatismo puede degradar el rendimiento en operaciones que involucran múltiples tablas relacionadas o lógica transaccional compleja: ? documentan empíricamente, comparando configuraciones de un framework ORM líder contra SQL manualmente optimizado, patrones de rendimiento indeseables (*anti-patrones*) que el uso no cuidadoso de un ORM introduce. Un procedimiento almacenado (*stored procedure*, SP), en contraste, ejecuta lógica SQL directamente dentro del motor de base de datos, con control explícito sobre el plan de ejecución. SGB-SaaS adopta una estrategia híbrida – ORM para CRUD elemental, SP para operaciones multi-tabla transaccionalmente sensibles – cuyo mecanismo concreto de invocación segura (parámetros nombrados bindeados, no concatenación de cadenas) se documenta con código real en el 6.

El patrón *cache-aside* (también llamado *lazy loading* de caché) consulta primero una caché de acceso rápido (en este proyecto, Redis) y solo recurre a la base de datos relacional cuando hay un fallo de caché (*cache miss*), momento en el que el resultado se escribe de vuelta en la caché para peticiones futuras. ? miden empíricamente el efecto de este patrón

– separando el tipo de objeto cacheado (contenido estático vía acelerador HTTP, resultados de consulta vía Redis) – documentando reducciones reales de carga sobre la base de datos relacional bajo tráfico concurrente. La instancia concreta de este patrón en SGB-SaaS (anotación `@Cacheable` sobre el catálogo de libros, con invalidación explícita vía `@CacheEvict`) y su evaluación empírica bajo carga (k6, comparación estadística `cache_caliente` vs. `cache_frio`) se desarrollan en el 5 y en el 4, respectivamente.

Materiales y métodos

Enfoque metodológico: Design Science Research

SGB-SaaS se desarrolló siguiendo, de forma implícita durante el proyecto y reconstruida aquí de forma explícita, la estructura de seis actividades de *Design Science Research* (DSR) de Peffers et al. (?) – un proceso iterativo pensado específicamente para investigación que produce un artefacto (en este caso, software) como resultado, no solo conocimiento descriptivo. La evaluación del artefacto se rige por las siete guías de Hevner et al. (?) para la investigación en diseño en sistemas de información, en particular la evaluación observacional del artefacto desplegado en su entorno real de uso. Las seis actividades se mapean así contra la evolución real y documentada del proyecto:

1. **Identificación del problema y motivación.** La fricción operativa real de la gestión bibliotecaria manual (1), delimitada en la fase de planificación del proyecto (Entrega 1A).
2. **Definición de objetivos de una solución.** Las cuatro preguntas de investigación (RQ1–RQ4) y los cinco objetivos específicos (OE1–OE5) declarados en el 1.
3. **Diseño y desarrollo.** Construcción incremental del artefacto en sucesivas entregas – Entrega 1B (primer módulo funcional), Entrega 3 (candidato estable con los ocho módulos y evidencia empírica inicial), Entrega Final (HEAD en `demo/interfaces-completas`, este documento) – con las decisiones de arquitectura formalizadas como ADR y el modelo C4 (5).
4. **Demostración.** Ejecución de casos de uso reales de punta a punta contra el stack Docker completo, no simulados: `docs/mediciones/backend/-07-29-flujo-prestamo-devolucion-multa-e2e.md` y `docs/mediciones/frontend/-07-30-1`.
5. **Evaluación.** El conjunto de instrumentos de medición descrito en la 4.2 de este mismo capítulo (k6, JaCoCo, Lighthouse, auditoría OWASP, y SUS pendiente de ejecución).
6. **Comunicación.** Este mismo informe académico, el checklist de calidad de requisitos INCOSE v4 (`docs/checklists/incose2023-req.md`), la documentación arquitectónica versionada como código (`docs/arquitectura/`), y el artefacto de software archivado con DOI persistente en Zenodo (1).

Instrumentos de medición

- **System Usability Scale (SUS)** (?): 10 ítems tipo Likert de 5 puntos, alternando afirmaciones positivas y negativas, que producen una puntuación agregada de 0 a 100 por participante mediante la fórmula estándar del instrumento. Es el instrumento del Bloque C.3 de esta guía; su estado de ejecución real – todavía pendiente – se declara con honestidad en la 4.4.
- **k6** (`k6/libros-listado-test.js`, `k6/opts.js`): herramienta de prueba de carga HTTP, usada para medir la latencia del endpoint de catálogo bajo 50 VUs concurrentes en los escenarios `cache_caliente/cache_frio`. Su protocolo completo se describe en la 4.5.
- **Lighthouse** (`docs/mediciones/lighthouse/lhci-20260731-0300.json`): auditoría automatizada de calidad del frontend, con puntuaciones reales obtenidas de 0,95 en Performance, 0,95 en Accessibility, 1,00 en Best Practices y 0,82 en SEO sobre una escala de 0 a 1.
- **JaCoCo** (`jacoco-maven-plugin` en `backend-springboot/pom.xml`): cobertura de línea/rama/complejidad ciclomática sobre el backend, con objetivo declarado de la guía de $\geq 70\%$ de líneas para la Entrega Final. El resultado de cierre vigente es **56,30 %** de líneas (2195/3899) y **30,83 %** de ramas (390/1265) — medición en fresco sobre `568e8c1f` (7.3) —, por debajo del umbral de líneas. Historia de la cifra: tras un primer ciclo de trabajo dirigido específicamente a cinco clases de autenticación/autorización que partían de cobertura casi nula, una medición intermedia que confirmó que el porcentaje subió a 81,64 % tras el merge de los ocho módulos de dominio nuevos que casi triplicaron la base de código analizable (`docs/mediciones/jacoco/-08-13-cobertura-jacoco-post-merge-8-modulos.md`), una corrida de cierre posterior que registró 56,01 % (1613/2880, `89eb6ab`) tras la incorporación de funcionalidad adicional, y finalmente la medición vigente de 56,30 % citada arriba (ver 7.3 para el detalle completo).

- **Auditoría OWASP** (`scripts/owasp-audit.sh`, `docs/mediciones/sec/`): verificación reproducible vía `curl` contra el stack Docker real de un subconjunto representativo de controles del OWASP Top 10:2021 (3.7), con hallazgo y corrección documentados por control.

Enfoque de métricas: Goal-Question-Metric (GQM)

El bloque de evaluación de rendimiento de SGB-SaaS se estructuró, retrospectivamente, según el patrón Goal-Question-Metric:

Meta (*Goal*). Analizar el uso de caché Redis sobre el endpoint de catálogo, con el propósito de evaluar su efecto sobre el rendimiento observable (latencia), desde el punto de vista del equipo de desarrollo, en el contexto de carga concurrente controlada (k6, 50 VUs).

Preguntas (*Questions*).

- Q1. ¿Cuál es la latencia p95 del endpoint con caché caliente frente a caché frío?
- Q2. ¿La diferencia observada entre ambos escenarios es estadísticamente significativa, y de qué magnitud es el efecto?
- Q3. ¿Se cumplen los umbrales de latencia exigidos por la guía para cada escenario?

Métricas (*Metrics*).

- M1. Percentiles p50/p90/p95/p99 de `http_req_duration` (ms) por escenario, sobre las 5 corridas agregadas (`scripts/perf-analysis.py`).
- M2. Estadístico y p -valor de la prueba de Wilcoxon de rangos con signo (pareada, sobre el p95 de cada corrida), y tamaño de efecto de Cliff's delta.
- M3. Cumplimiento binario (sí/no) de los umbrales exigidos: p95 <200 ms en caliente, p95 <500 ms en frío.

Los resultados reales obtenidos para M1–M3 están en `docs/mediciones/perf/REPORT.md` y se transcriben en el 5. Este mismo patrón GQM podría extenderse a los otros bloques de evaluación (seguridad, cobertura, usabilidad); no se desarrolla aquí para cada uno por disciplina de alcance de este capítulo – un GQM completo es suficiente para documentar el enfoque de métricas adoptado, sin redundar en una plantilla repetida para cada bloque.

Población, muestreo y participantes

La población objetivo real de este proyecto es la comunidad de la Universidad Técnica Estatal de Quevedo (UTEQ) en sus roles de dominio (lector, bibliotecario) – no una muestra sintética ni un panel externo contratado. ¿, en su revisión crítica de prácticas de muestreo en investigación de ingeniería de software, advierten que muestras pequeñas y no aleatorizadas – el perfil esperable de cualquier estudio de este alcance, reclutado por conveniencia dentro de una única institución – limitan severamente la generalización de cualquier conclusión más allá de la muestra concreta estudiada, y recomiendan declarar explícitamente el método de reclutamiento en vez de presentar el resultado como representativo de una población más amplia.

Nota de honestidad – estado real de ejecución. La muestra SUS **no se ha reclutado ni ejecutado todavía** al momento de escribir este capítulo (OBS-08 en `docs/observaciones/OBSERVACIONES.md`: “Pendiente – en progreso, asignada a Panama”). Lo que sí existe, ya definido y versionado antes de la primera ejecución real (no de forma retroactiva), es el protocolo completo: plantilla de consentimiento informado (`docs/etica/consentimientos/plantilla.md`), técnica de muestreo por conveniencia dentro de la comunidad UTEQ, tamaño mínimo de muestra $N \geq 15$ participantes exigido por la guía, y anonimización mediante código de participante (P01, P02, ..., ver `docs/etica/ETHICS.md`, sección iii) – nunca nombre ni correo. No se declara una muestra ya obtenida para no incurrir en el mismo tipo de afirmación no verificada que este documento evita de forma sistemática en el resto de sus capítulos.

Protocolo experimental de rendimiento (replicable)

El bloque de rendimiento sigue un protocolo completamente reproducible, documentado con el comando exacto ejecutado:

```
docker run --rm --network sgb-saas_default \
-v "$(pwd)/k6:/scripts" -v "$(pwd)/docs/mediciones/perf:/out" \
grafana/k6 run --out json=/out/k6-run{N}.json /scripts/libros-listado-test.js
```

Se ejecutó 5 veces de forma independiente (el mínimo exigido por la guía), cada corrida cubriendo ambos escenarios de forma secuencial (`cache_caliente`: siempre `page=0`, para forzar repetidos aciertos de caché sobre la misma clave; `cache_frio`: página elegida por un PRNG determinista (`mulberry32`, semilla fija 42) en cada petición, para forzar fallo de caché en cada una), con un perfil de carga común de 10s de *ramp-up* a 50 VUs, 30s sostenidos, 10s de *ramp-down* (`k6/opts.js`). Autenticación real vía `POST /api/auth/login` en el `setup()` del script, reutilizando el mismo `accessToken` entre VUs. Los cinco archivos de datos crudos (`k6-run1.json`–`k6-run5.json`, formato NDJSON de k6) están versionados en `docs/mediciones/perf/`, permitiendo reanalizar los mismos datos sin repetir la carga. El detalle completo – entorno exacto (versiones de Docker, Java, PostgreSQL, Redis, k6), resultados por corrida y agregados – está en `docs/mediciones/perf/REPORT.md` y se resume en el 5.

Análisis estadístico

El análisis agregado de las 5 corridas (`scripts/perf-analysis.py`) calcula, por escenario, sobre la métrica `http_req_duration`: la media, la mediana, la desviación típica (σ), el intervalo de confianza al 95 % de la media, los percentiles p50/p90/p95/p99, la tasa de error HTTP ≥ 500 y el *throughput* (peticiones/segundo). Sobre la comparación pareada `cache_caliente` vs. `cache_frio` (p95 de cada una de las 5 corridas, no el *pool* de peticiones agregado, precisamente porque las corridas son las mismas 5 sesiones de carga y no muestras independientes) se aplica la prueba de Wilcoxon de rangos con signo (método exacto, vía `scipy.stats.wilcoxon`) y el tamaño de efecto de Cliff's delta. Los resultados reales obtenidos – Wilcoxon $p = 0,0625$, Cliff's delta = $-0,68$ – y su interpretación correcta en función de la potencia estadística del tamaño de muestra mínimo se desarrollan en el 12 (validez de conclusión).

Determinismo y semillas

Para que los resultados sean reproducibles bit a bit donde el diseño experimental usa aleatoriedad, se fijaron semillas explícitas en los dos únicos puntos del repositorio donde se generan números pseudoaleatorios con fines de medición (verificado por búsqueda directa en el código):

- **Selección de página en el escenario `cache_frio`** (`k6/libros-listado-test.js`): PRNG `mulberry32` con semilla fija **42**, de modo que la secuencia de páginas pedidas en cada corrida es la misma si se re-ejecuta el script.
- **Intervalo de confianza al 95 % por *bootstrap* del percentil p95** en el gráfico comparativo (`scripts/perf-analysis.py`, `BOOTSTRAP_SEED = 42`): 2000 réplicas de remuestreo con reemplazo, con la misma semilla **42** – un percentil, a diferencia de la media, no tiene una fórmula cerrada de intervalo de confianza, de ahí el uso de *bootstrap*.

Ninguna otra parte del repositorio (scripts de prueba, seed de base de datos, generación de datos de ejemplo) usa aleatoriedad con fines de medición que requiera declarar una semilla adicional.

Diseño y arquitectura del sistema

El modelo arquitectónico de SGB-SaaS, aplicando el modelo C4 descrito en la 3.3, está versionado como código fuente en `docs/arquitectura/workspace.dsl` (Structurizr DSL, modelo C4), en sus tres niveles completos – Contexto, Contenedores y Componentes – que reemplazan los diagramas estáticos sin fuente versionada de la Entrega 1A (`docs/diagramas/diagrama_c4_capa1.png`, `diagrama_c4_capa2.jpeg`). Esos dos archivos permanecen versionados en el repositorio como evidencia histórica del diseño original, pero ya no se reproducen en este documento porque estaban desactualizados (citaban Google Books API como sistema externo, sin integración real en el código, y el segundo no incluía el contenedor Redis). Las figuras 5.1, 5.2 y 5.4 son la exportación real y automática de `workspace.dsl` con Structurizr ([`sec:c4-exportacion`]), no una reconstrucción manual.

Nivel 1 – Contexto del sistema

Cinco actores interactúan con SGB-SaaS como caja negra: **Lector** (consulta el catálogo, reserva libros, ve su historial de préstamos y multas), **Bibliotecario** (registra préstamos, devoluciones y multas desde el mostrador; gestiona el catálogo), **Gerente** (supervisa la operación: cuentas de personal, catálogos maestros de estado y reportes de auditoría), **Administrador** (configura parámetros del sistema y catálogos base de la plataforma – rol que no existía en el diseño original de Entrega 1A, incorporado al normalizar el modelo RBAC) y **Visitante anónimo** (sin cuenta todavía; hoy solo puede registrarse o iniciar sesión).

El propio `workspace.dsl` documenta, en su comentario de cabecera, una discrepancia real entre el diseño original de Entrega 1A y la implementación actual que vale la pena señalar aquí: Gemini 3.5 Flash Lite API y Google Books API aparecían como sistemas externos en el diseño original y se **retiraron** del diagrama actual porque no existe ninguna

integración con Google Books en el código real (Gemini sí se integró, pero como parte interna del contenedor Backend, no como un "sistema externo" del Nivel 1 – ver 5.3 más abajo). Tampoco se diseñó nunca un endpoint de catálogo público accesible sin autenticación: **LibroController** exige rol **LECTOR** (o superior) incluso para listar libros, así que el alcance real del *Visitante anónimo* se limita a registro e inicio de sesión.

Nivel 2 – Contenedores

Cuatro contenedores, alineados 1:1 con `docker-compose.yml`:

- **Frontend Angular** (TypeScript/Angular 21) – SPA de catálogo, formularios reactivos y gestión de sesión con el `accessToken` en memoria, nunca en `localStorage`.
- **Backend Spring Boot** (Java 21/Spring Boot 4.0.6) – API REST con autenticación JWT (cookie `HttpOnly` para el `refreshToken`), autorización RBAC por roles, y lógica de negocio de préstamos/reservas/multas vía JPA y procedimientos SQL.
- **PostgreSQL 16** – usuarios, roles/permisos, catálogo, préstamos, reservas, multas y bitácora de auditoría (26 tablas).
- **Redis 7** – blacklist de tokens JWT revocados (logout / revocación) y caché del listado de libros con TTL configurable externamente.

El diagrama entidad-relación real del esquema PostgreSQL (5.3) se generó automáticamente aplicando, en orden numérico, las 35 migraciones Flyway de `database/migrations/` (V1 a V36, con el hueco ya documentado en V29) sobre una instancia PostgreSQL 16 real y limpia, y extrayendo el esquema resultante con `eralchemy2`. El resultado tiene 44 tablas – más que las 26 citadas en la enumeración de arriba, cifra que no se corrige aquí por quedar fuera del alcance de esta tarea puntual; ver nota en el reporte de esta sesión.

Respecto al diseño original de Entrega 1A, el contenedor Redis y el actor Administrador son incorporaciones directas de la normalización RBAC y del mecanismo de revocación de tokens vía *blacklist*, ambos ausentes del diagrama de contenedores original.

Nivel 3 – Componentes

El Nivel 3 se completó una vez cerradas las 8 ramas del módulo de Préstamos/Reservas/Multas (evitando así reconstruir el diagrama dos veces mientras ese módulo seguía en desarrollo activo). Se construyó a partir del inventario real de *controllers/services/clases* de seguridad del backend – no del diseño aspiracional –, agrupado por dominio en 21 componentes para mantener el diagrama legible en vez de tener una entrada por cada una de las cerca de 30 clases reales:

- **Capa API** (7 componentes): Auth API, Catálogo API, Préstamos API, Notificaciones API, CredencialQR API, Chatbot API, Admin API – un componente por dominio funcional, no por clase `@RestController` individual.
- **Capa de servicio** (8 componentes): Auth Service, Catálogo Service, Préstamos Service, Reportes Service, Notificaciones Service, CredencialQR Service, Chatbot Service, Admin Service.
- **Seguridad** (5 componentes): `JwtService`, `JwtAuthFilter`, `UserDetailsServiceImpl`, `LoginRateLimiter` (OWASP A07, límite de intentos de login por correo+IP) y `ChatbotRateLimiter` (límite de mensajes al asistente por usuario).
- **Integración externa** (1 componente): `GeminiClient` – cliente HTTP hacia la API de Gemini 3.5 Flash Lite, con reintento simple ante 429/timeout/5xx y sin exponer la URL con la API key en logs (ver ADR-016, 5.4).

Las relaciones entre componentes se verificaron contra el código real (no se infirieron): cada flecha API → Service, Service → Postgres/ Redis, y las dependencias cruzadas de `ChatbotService` hacia `CatálogoService` y `PréstamosService` para el *grounding* real de sus respuestas (consulta disponibilidad de libros y reservas del propio usuario antes de responder, para no inventar disponibilidad) corresponden a llamadas reales entre las clases Java correspondientes.

Validación sintáctica y exportación visual

El archivo se validó sintácticamente con el motor activamente mantenido `structurizr/structurizr` (subcomando `validate`), no con `structurizr/cli` – que en este momento del ecosistema Structurizr es una imagen retirada que solo imprime un aviso de desuso y termina con código de salida 0 para cualquier entrada, válida o no (confirmado también al intentar el `export` con esa imagen: código de salida 0 sin generar ningún archivo), por

lo que una validación o exportación con esa imagen no habría sido real. Los tres niveles se exportaron con el mismo flujo activamente mantenido: `structurizr/structurizr export -workspace workspace.dsl -format plantuml`, seguido de renderizado con PlantUML (`plantuml/plantuml` vía Docker). El diagrama de Nivel 3 resultó en `docs/arquitectura/c4-nivel3-componentes.svg` (generado en una sesión previa; se confirmó en esta sesión que sigue coincidiendo con el `workspace.dsl` vigente – mismos 21 componentes, mismas relaciones – por lo que no fue necesario regenerarlo), versionado junto al DSL fuente y reproducido en la 5.4. Los diagramas de Nivel 1 y Nivel 2 se exportaron en esta sesión de la misma forma, resultando en `docs/arquitectura/c4-nivel1-contexto.png` y `c4-nivel2-contenedores.png`, reproducidos en la 5.1 y la 5.2.

Decisiones arquitectónicas documentadas (ADR)

El repositorio contiene **13 Architecture Decision Records** (`docs/adr/`), agrupados a continuación según los 6 temas obligatorios del Bloque D de la guía (mapeo completo en `docs/adr/README.md`). Hasta el 26 de agosto de 2026, dos de esos temas estaban cubiertos por un ADR cuya numeración interna del repositorio no coincidía con la numeración sugerida por la guía (ADR-006/ADR-007 sugeridos por la guía para acceso a datos y despliegue correspondían a ADR-013 y ADR-012 en este repositorio), por continuidad con la numeración ya evaluada en la Tercera Entrega. Se decidió reenumerar de todas formas para que la numeración coincida literalmente con la guía en vez de dejarlo como una diferencia solo explicada en prosa – ver la nota histórica en `docs/adr/README.md` con el detalle del intercambio y las referencias cruzadas actualizadas en el mismo cambio.

#	Tema obligatorio (Bloque D)	ADR(s)
1	Elección de la pila tecnológica principal	ADR-001
2	Esquema de autenticación	ADR-010 + ADR-003 + ADR-012
3	Gestor de base de datos	ADR-011
4	Estrategia de caché	ADR-008
5	Estrategia de despliegue	ADR-007
6	Estrategia de acceso a datos (CRUD-ORM vs. SP)	ADR-006

Los 7 ADR restantes no mapean a ninguno de los 6 temas obligatorios, pero documentan decisiones reales tomadas durante el desarrollo: ADR-013 (versionado de esquema reproducible, Flyway + snapshot), ADR-009 (licencia MIT), ADR-014 (separación de responsabilidades entre ADMIN y GERENTE: el primero administra permisos/configuración de plataforma, el segundo la operación diaria – ninguno de los dos queda excluido de *ver* el padrón de usuarios, pero solo ADMIN puede *modificarlo*), ADR-015 (TLS termina en el proxy, no en el backend Spring Boot: el backend queda preparado vía `server.forward-headers-strategy: framework` para reconocer X-Forwarded-Proto y activar HSTS el día que el proxy real active TLS, sin cambios de código adicionales), y ADR-016 (qué datos exactos viajan a la API externa de Gemini en el chatbot – texto del mensaje, historial de la sesión y un prompt de *grounding* con datos ya públicos del catálogo – y por qué eso no constituye una exposición indebida de información: nunca viajan credenciales, tokens, contraseñas ni identificación personal).

Atributos de calidad (ISO/IEC 25010)

Aplicando el modelo de calidad descrito en la 3.4, `docs/arquitectura/ISO25010.md` prioriza las 8 características de calidad de producto de la norma para el contexto específico de este proyecto (biblioteca universitaria de bajo volumen, no un sistema de alta concurrencia), citando evidencia real donde existe en vez de estimarla. Se reproduce a continuación tal cual el documento fuente (única fuente de verdad de esta tabla; este capítulo no la duplica de memoria):

Característica	Prioridad	Estrategia / decisión que la atiende
Adecuación funcional	Alta	Procedimientos almacenados transaccionales para operaciones con lógica cruzada multi-tabla (<code>sp_registrar_devolucion</code> ,
Eficiencia de desempeño	Media	Caché Redis del listado de libros con TTL externo (ADR-008). Evidencia vigente: prueba de carga formal (50 VUs, 5 co
Compatibilidad	Media	API REST documentada con OpenAPI/Swagger vía <code>springdoc-openapi</code> (ADR-001), JSON como formato estándar – sin
Usabilidad	Alta	<code>sp_registrar_devolucion</code> valida el estado del préstamo antes de actuar (rechaza con LB409 si ya estaba devuelto, evitand
Fiabilidad	Alta	Parcialmente atendida: ADR-003 documenta explícitamente que, si Redis cae, <code>JwtAuthFilter</code> queda sin forma de verifica
Seguridad	Alta	Cookie <code>HttpOnly+Secure+SameSite=None</code> para el <code>refreshToken</code> (ADR-012), BCrypt costo 12, blacklist de tokens revocados
Mantenibilidad	Alta	Los ADR de <code>docs/adr/</code> documentan decisiones no obvias con sus alternativas consideradas; <code>docs/basedatos/CATALOGO-SP.</code>
Portabilidad	Alta	Docker Compose con imágenes base pinadas por <code>digest sha256, db/init/01-consolidado.sql</code> generado de forma reproduc

¹ `SameSite=None` era necesario para que el navegador enviara la cookie entre los orígenes cross-origin de Render (frontend y backend en subdominios distintos), pero reintroduce la exposición a CSRF que `SameSite=Strict` bloqueaba por sí solo. El código actual tiene `.csrf(csrf -> csrf.disable())` activo (`SecurityConfig.java`, línea 55) y no implementa ningún token CSRF; el único control vigente es la lista blanca de CORS, que impide leer la respuesta cross-origin pero no impide el envío de la petición con la cookie adjunta. El ADR-012 documenta esto explícitamente como riesgo aceptado, no mitigado, con la corrección (token CSRF / *double-submit cookie*) registrada como trabajo futuro (10.5).

Matriz de trazabilidad – resumen por tipo de acceso a datos

La matriz completa de los 46 requisitos vive en `docs/trazabilidad/matriz.csv` y se valida automáticamente en cada ejecución de CI vía `scripts/validate-traceability.sh`. Este capítulo no reproduce las 46 filas (el detalle completo por requisito vive en 9 y en la propia matriz); resume aquí la columna `tipo_acceso`, relevante para esta sección por ser la evidencia directa de que la estrategia híbrida CRUD-ORM/SP declarada en ADR-006 se aplica de verdad y no solo en teoría.

Nota de corrección respecto a una versión anterior de este capítulo. `matriz.csv` tenía 4 filas (REQ-F-018, REQ-F-019, REQ-F-020, REQ-F-028) con una coma sin escapar dentro de un campo, que rompía el parseo CSV estándar – defecto ya corregido en el commit `ecfaf52`. Una versión anterior de esta tabla, calculada antes de esa corrección, solo había detectado 2 de las 4 filas afectadas (REQ-F-019, REQ-F-020) y las marcaba como “sin clasificar”; además atribuía el problema de REQ-F-019 a la columna `tipo_acceso`, cuando en realidad ese campo sí tenía un valor de `tipo_acceso` válido (“CRUD-ORM + generacion de imagen (ZXing)”) – el campo roto en esa fila específica era `modulo_codigo`, no `tipo_acceso`. Con las 43 filas parseando limpio, la tabla de abajo reclasifica las 4 filas correctamente en vez de dejarlas fuera:

Tipo de acceso a datos	Requisitos	%
CRUD-ORM (exclusivo, incl. combinado con Redis/SMTP/caché/API externa)	21	48,8 %
Híbrido SP + CRUD-ORM (requisitos transversales)	2	4,7 %
Procedimiento/función SQL (SP, exclusivo, incl. combinado con generación de PDF)	8	18,6 %
No aplica (decisión de arquitectura, configuración, control de acceso o UI sin acceso directo a BD)	11	25,6 %
Redis + SMTP (sin tabla Postgres nueva; no aplica CRUD-ORM ni SP) ³	1	2,3 %
Total	43	100,0 %

Nota de alcance (cierre de la Entrega Final). La matriz creció de 43 a 46 requisitos tras esta tabla (REQ-F-030, REQ-F-031, y una extensión de REQ-F-021 a la capa frontend) – los 3 no se reclasificaron en la tabla de arriba porque la categorización por `tipo_acceso` exige revisar el valor completo de cada campo a mano, no solo una coincidencia de texto (ver el caso de REQ-F-018 más abajo, que una primera pasada automática clasificó mal por esa misma razón); no hubo tiempo de repetir esa revisión manual para los 3 antes del cierre. Los porcentajes de esta tabla y del párrafo siguiente siguen describiendo, con honestidad, la distribución de los 43 requisitos originales, no los 46 vigentes.

De los 43 requisitos originales de esta tabla, 10 (23,3%) involucran al menos un procedimiento o función SQL (8 exclusivamente, 2 en combinación con CRUD-ORM: REQ-NF-004, transversal a los módulos de préstamos/multas/reservaciones vs. auth/libros/bitácora, y REQ-NF-013, prevención de inyección SQL sobre ambos mecanismos), consistente con la justificación de ADR-006: reservar los procedimientos almacenados para operaciones con validación cruzada multi-tabla o transacciones atómicas compuestas (registrar préstamo, registrar devolución, pagar multa, anular multa, expirar reservaciones vencidas, y los 3 reportes agregados), dejando el CRUD elemental de una sola tabla en Spring Data JPA. REQ-F-018 (renovación de préstamo), pese a mencionar textualmente “SP” en su valor de `tipo_acceso` (“CRUD-ORM (validación en Java, sin SP nuevo)”), es CRUD-ORM exclusivo: la frase aclara explícitamente que la renovación *no* introdujo un procedimiento almacenado nuevo, la validación de sus 3 reglas de negocio corre en Java – se señala aquí porque una primera pasada de clasificación automática por coincidencia de texto la contó por error como híbrida, hasta revisar el valor completo del campo.

Implementación

Este capítulo documenta decisiones críticas de implementación del backend, cada una con al menos un listado de código real extraído directamente del repositorio.

Procedimiento almacenado y su invocación desde Spring Data JPA

La estrategia híbrida de acceso a datos (fundamentada en la 3.8; ADR-006, 5.4) exige que las operaciones con validación cruzada multi-tabla se implementen como procedimiento o función SQL. El siguiente listado es `db/procs/sp_crear_prestamo.sql` completo: registra un préstamo de forma atómica, validando que el usuario no esté

²El texto original de `IS025010.md` en esta fila cita “6 ADRs existentes”, cifra que quedó desactualizada tras incorporar los 8 módulos construidos después de la Tercera Entrega – el repositorio contiene 13 ADRs a la fecha de este capítulo (5.4). Se transcribe el texto original de la fuente junto con esta aclaración en vez de editarlo silenciosamente, ya que corregir `IS025010.md` está fuera del alcance de esta tarea.

³REQ-F-020: `tipo_acceso` = “Redis (código OTP con TTL, sin tabla nueva) + SMTP (EmailService)”. Categoría propia, no un defecto de la matriz – el código de verificación vive en Redis con TTL, sin persistirse en una tabla Postgres nueva, y el envío usa SMTP directo; no encaja limpiamente en CRUD-ORM ni en SP.

bloqueado por multas y que el libro tenga stock disponible, usando SQLSTATE personalizados (LB404/LB422) para que el backend traduzca el error sin parsear texto (ver Listado [lst:sp-crear-prestamo]).

```
CREATE OR REPLACE FUNCTION sp_crear_prestamo(
    p_usuario_id BIGINT,
    p_libro_id BIGINT,
    p_bibliotecario_id BIGINT,
    p_dias_prestamo INTEGER
)
RETURNS BIGINT
LANGUAGE plpgsql
AS $$
DECLARE
    v_estado_usuario_id    INTEGER;
    v_estado_bloqueado_id  INTEGER;
    v_stock_disponible      SMALLINT;
    v_estado_activo_prestamo_id INTEGER;
    v_prestamo_id          BIGINT;
BEGIN
    SELECT id INTO v_estado_bloqueado_id
    FROM estados_usuario
    WHERE nombre = 'BLOQUEADO_POR_MULTA';

    SELECT estado_id INTO v_estado_usuario_id
    FROM usuarios
    WHERE id = p_usuario_id
    FOR UPDATE;

    IF NOT FOUND THEN
        RAISE EXCEPTION 'El usuario % no existe', p_usuario_id
        USING ERRCODE = 'LB404';
    END IF;

    IF v_estado_usuario_id = v_estado_bloqueado_id THEN
        RAISE EXCEPTION 'El usuario % esta bloqueado por multas pendientes ...',
        p_usuario_id USING ERRCODE = 'LB422';
    END IF;

    SELECT stock_disponible INTO v_stock_disponible
    FROM libros WHERE id = p_libro_id FOR UPDATE;

    IF NOT FOUND THEN
        RAISE EXCEPTION 'El libro % no existe', p_libro_id
        USING ERRCODE = 'LB404';
    END IF;

    IF v_stock_disponible <= 0 THEN
        RAISE EXCEPTION 'El libro % no tiene stock disponible', p_libro_id
        USING ERRCODE = 'LB422';
    END IF;

    UPDATE libros SET stock_disponible = stock_disponible - 1
    WHERE id = p_libro_id;

    INSERT INTO prestamos (usuario_id, libro_id, bibliotecario_id,
        fecha_prestamo, fecha_devolucion_estimada, estado_prestamo_id)
    VALUES (p_usuario_id, p_libro_id, p_bibliotecario_id, NOW(),
        NOW() + make_interval(days => p_dias_prestamo),
        v_estado_activo_prestamo_id)
```

```

RETURNING id INTO v_prestamo_id;

RETURN v_prestamo_id;
END;
$$;

```

Nota de honestidad sobre el mecanismo de invocación. La guía de esta entrega espera que el método Java invoque el procedimiento con la anotación `@Procedure` de Jakarta Persistence. En este repositorio, ese *fue* el mecanismo original, pero se reemplazó por `@Query(nativeQuery = true)` tras un fallo real en tiempo de ejecución: Hibernate genera sintaxis de parámetros nombrados de PostgreSQL (`nombre => valor`) dentro del escape JDBC `{call ...}` usado por `@Procedure`, que el driver `pgjdbc` no soporta ahí – `PrestamoMultiProcedureIntegrationTest` (test de integración real contra PostgreSQL, no mocks) confirmó el fallo en la primera ejecución. La migración de los 5 métodos afectados está documentada en `docs/mediciones/backend/2026-07-28-fallo-invocacion-sp-multi-out.md` y referenciada como *issue* conocido de Spring Data JPA (`spring-projects/spring-data-jpa#3393`). El siguiente listado muestra el método real vigente hoy en `PrestamoProcedureRepository.java`, incluyendo el comentario del propio código que documenta por qué ya no usa `@Procedure`:

```

/**
 * sp_crear_prestamo: retorno escalar unico (BIGINT). Antes usaba
 * @Procedure(procedureName=...) -- fallaba en runtime con
 * "syntax error at or near '=>'" porque Hibernate genera sintaxis de
 * parametros nombrados de PostgreSQL dentro del escape JDBC
 * {call ...}, que pgjdbc no soporta. @Query nativa con SELECT evita
 * el CallableStatement por completo.
 */
@Query(value = "SELECT sp_crear_prestamo(:p_usuario_id, :p_libro_id, " +
              ":p_bibliotecario_id, :p_dias_prestamo)",
      nativeQuery = true)
Long spCrearPrestamo(
    @Param("p_usuario_id") Long usuarioId,
    @Param("p_libro_id") Long libroId,
    @Param("p_bibliotecario_id") Long bibliotecarioId,
    @Param("p_dias_prestamo") Integer diasPrestamo
);

```

Los parámetros siguen bindeándose por nombre (`:p_usuario_id`, etc.) exactamente igual que con `@Procedure` – la única diferencia es el mecanismo JDBC que usa Hibernate para transportar la llamada (`PreparedStatement` en vez de `CallableStatement`), por lo que la prohibición de SQL dinámico o concatenación de entrada de usuario (regla A.2.3 de la guía) se sigue cumpliendo igual (`docs/basedatos/CATALOGO-SP.md`, sección “Nota de diseño: por qué 5 usan `@Procedure`...y 2 usan `@Query`”). De los 7 objetos SQL del catálogo, los 5 con efectos secundarios usaban originalmente `@Procedure`/ `@NamedStoredProcedureQuery` y hoy usan `@Query(nativeQuery = true)` tras esta migración; las 2 funciones de solo lectura que retornan `TABLE` (varias filas) usaron `@Query` nativa desde el principio, porque la API de *stored procedures* de JPA 2.1 no tiene una forma estándar de exponer un `RETURNS TABLE` de PostgreSQL a través de `@Procedure`.

¿Esto viola la prohibición de SQL dinámico de la guía (A.2.1)?

No. La guía (Bloque A.2.1) establece textualmente que “queda prohibido invocarlos mediante concatenación de cadenas en `createNativeQuery(...)`”. Esa prohibición apunta a un patrón concreto y distinto del usado en este proyecto: construir la sentencia SQL en tiempo de ejecución **concatenando texto de entrada del usuario** dentro del propio literal de la consulta – p. ej. `createNativeQuery("SELECT sp_crear_prestamo(" + idUsuario + ")")`, donde `idUsuario` pasa a formar parte literal de la cadena SQL antes de ejecutarse, el vector clásico de inyección SQL.

El código real de este proyecto **no hace eso en ningún punto**. El listado [lst:sp-crear-prestamo-java] usa `@Query(value = "SELECT sp_crear_prestamo(:p_usuario_id, ...)", nativeQuery = true)` con **parámetros nombrados bindeados** (`:p_usuario_id`, `:p_libro_id`, ...) resueltos por Spring Data JPA/Hibernate vía `PreparedStatement` antes de llegar al driver JDBC – el valor de cada parámetro nunca se concatena, formatea ni interpola dentro del literal de la cadena SQL; el literal que Java compila es fijo (`"SELECT sp_crear_prestamo(:p_usuario_id, :p_libro_id, :p_bibliotecario_id, :p_dias_prestamo)"`) y el binding ocurre por separado, exactamente como con `@Procedure`. Esta forma de `@Query` nativo con parámetros nombrados es, en sí misma, un mecanismo formal y documentado de

Spring Data JPA para invocar funciones/procedimientos parametrizados – no es un atajo informal ni una forma disimulada de SQL dinámico.

En síntesis: la regla A.2.1 prohíbe construir SQL concatenando entrada de usuario dentro del texto de la consulta; los parámetros bindeados de `@Query` nativa no construyen SQL a partir de input en absoluto, por lo que este patrón no cae bajo esa prohibición. El motivo del cambio desde `@Procedure`/`@NamedStoredProcedureQuery` tampoco fue estilístico: está documentado con evidencia real y verificable en docs/mediciones/backend/2026-07-28-fallo-invocacion-sp-multi-out.md (fallo reproducido en runtime, con el mensaje de error real de PostgreSQL/pgjdbc) y corresponde a un defecto conocido de la combinación Hibernate 6.2+/7.x con procedimientos multi-OUT en PostgreSQL, reportado en el repositorio público de Spring Data JPA (spring-projects/spring-data-jpa#3393) – no una decisión arbitraria del equipo para eludir el mecanismo que pide la guía.

Nota sobre verificación automática. Al momento de escribir este capítulo no existe todavía en el repositorio un script de análisis estático tipo `scripts/audit-sql-dynamic.sh` que detecte patrones de SQL dinámico en el código fuente – se deja constancia aquí para que, cuando se incorpore, su regla de detección distinga explícitamente entre concatenación real de entrada de usuario dentro del texto de una consulta (el patrón que sí debe marcar) y `@Query(nativeQuery = true)` con parámetros nombrados bindeados (`:p_...`) como el de esta sección, que no debería marcarse como hallazgo – un análisis ingenuo basado solo en buscar la cadena `nativeQuery = true` sin inspeccionar si el valor de `value` contiene variables Java concatenadas produciría un falso positivo contra este patrón, ya verificado como seguro.

Manejo uniforme de errores (RFC 7807)

Todas las respuestas de error del backend se devuelven como `ProblemDetail` (RFC 7807), vía un único `@RestControllerAdvice` (`GlobalExceptionHandler`). Un caso particularmente representativo es la traducción de los `SQLSTATE` personalizados de los procedimientos almacenados (6.1) a códigos HTTP, sin que el backend tenga que parsear el texto del mensaje de error (ver Listado [lst:global-exception-handler]):

```
@ExceptionHandler({
    InvalidDataAccessResourceUsageException.class,
    UncategorizedSQLException.class,
    DataAccessException.class
})
public ProblemDetail handleStoredProcedureError(Exception ex) {
    Throwable causa = ex;
    while (causa != null && !(causa instanceof SQLException)) {
        causa = causa.getCause();
    }

    if (causa instanceof SQLException sqlEx) {
        String sqlState = sqlEx.getSQLState();
        HttpStatus status = switch (sqlState) {
            case "LB404" -> HttpStatus.NOT_FOUND;
            case "LB409" -> HttpStatus.CONFLICT;
            case "LB422" -> HttpStatus.UNPROCESSABLE_ENTITY;
            default -> null;
        };
        if (status != null) {
            return ProblemDetail.forStatusAndDetail(status, sqlEx.getMessage());
        }
    }

    log.error("Error no controlado en procedimiento almacenado", ex);
    return ProblemDetail.forStatusAndDetail(
        HttpStatus.INTERNAL_SERVER_ERROR, "Error interno del servidor");
}
```

Sin este manejador dedicado, cualquier `RAISE EXCEPTION ... USING ERRCODE = 'LBxxx'` de un procedimiento llegaba envuelto en una `DataAccessException` genérica de Spring, y caía en el *catch-all* de último recurso – un error de negocio real (p. ej. “préstamo ya devuelto”) se veía como un 500 genérico en vez de un 409 con el mensaje real. El

mismo patrón de un `@ExceptionHandler` dedicado por tipo de excepción se repite para excepciones de dominio Java (p. ej. `CorreoYaRegistradoException` → 409, `LoginRateLimitExcedidoException` → 429) y para excepciones del propio framework que antes caían también en el *catch-all*: `AuthorizationDeniedException` (autorización de método vía `@PreAuthorize`, antes producía 500 en vez de 403) y `NoResourceFoundException` (recurso estático inexistente bajo el perfil prod, corregido en esta misma entrega – ver docs/mediciones/sec/2026-08-11-owasp-a05-verificacion-real.md).

Configuración paramétrica del sistema

`ConfiguracionSistemaService` expone parámetros de la tabla `configuracion_sistema` (p. ej. el máximo de renovaciones de un préstamo, ver REQ-F-018) para leerse en cada operación de negocio sin ir a la base de datos cada vez, con un caché en memoria simple (`ConcurrentHashMap` por instancia, no Redis) invalidado clave por clave al escribir (ver Listado [lst:config-service]):

```
private final ConcurrentHashMap<String, String> cache = new ConcurrentHashMap<>();

@Transactional(readOnly = true)
public String obtenerValor(String clave) {
    String cacheado = cache.get(clave);
    if (cacheado != null) {
        return cacheado;
    }
    String valor = repo.findById(clave)
        .orElseThrow(() -> new EntityNotFoundException(CLAVE_NO_ENCONTRADA + clave))
        .getValor();
    cache.put(clave, valor);
    return valor;
}

@Transactional
public ConfiguracionSistemaResponseDTO actualizar(String clave, String nuevoValor) {
    ConfiguracionSistema config = repo.findById(clave)
        .orElseThrow(() -> new EntityNotFoundException(CLAVE_NO_ENCONTRADA + clave));
    config.setValor(nuevoValor);
    repo.save(config);
    cache.remove(clave);
    return new ConfiguracionSistemaResponseDTO(config.getClave(), config.getValor());
}
```

La elección deliberada de un `ConcurrentHashMap` local en vez de Redis (que sí se usa para el caché del catálogo, ver 6.4) está documentada en el propio Javadoc de la clase: este valor cambia con muy poca frecuencia (un ADMIN ajustando un parámetro puntual), a diferencia del caché “libros”, que sí necesita compartirse entre instancias e invalidarse por TTL. Además, `actualizar` solo permite modificar claves **ya existentes** – el catálogo de parámetros válidos lo define el dominio (migraciones/*seed*), no quien llama al endpoint, evitando que quede una clave suelta sin ningún consumidor real.

Estrategia de caché Redis del catálogo

El listado de libros usa caché declarativo de Spring (`@Cacheable/@CacheEvict`) sobre Redis, con TTL configurable externamente (ADR-008, ver Listado [lst:libro-service-cache]):

```
@Cacheable("libros")
@Transactional(readOnly = true)
public Page<LibroResponseDTO> listar(Pageable pageable) {
    return libroRepo.findByEstado_Nombre(ESTADO_ACTIVO, pageable)
        .map(this::toDTO);
}

@CacheEvict(value = "libros", allEntries = true)
@Transactional
public LibroResponseDTO crear(LibroRequestDTO dto) {
```

```

    if (libroRepo.existsByIsbn(dto.isbn())) {
        throw new IllegalArgumentException("ISBN ya registrado: " + dto.isbn());
    }
    validarStock(dto.stockTotal(), dto.stockDisponible());
    return toDTO(libroRepo.save(fromDTO(dto)));
}

```

crear, actualizar y eliminar invalidan el caché completo del listado (`allEntries = true`) para no dejar una respuesta paginada obsoleta tras cualquier escritura. El método `listarPorCategoria` (filtro por categoría/autor) deliberadamente **no** lleva `@Cacheable`: combinar el filtro con el caché exigiría una clave compuesta fuera del alcance de la rama que lo introdujo, y no es el camino más transitado del catálogo – decisión documentada como comentario en el propio código, no un descuido. El método `sugerir` (autocompletado) usa un caché *distinto* ("`sugerencias-libros`") con TTL propio, mucho más corto (5–10s), porque su patrón de invalidación (una entrada por texto de búsqueda) es incompatible con el caché de listado general.

Transporte del token de sesión en cookie `HttpOnly`

El `refreshToken` viaja exclusivamente en una cookie `HttpOnly`, `Secure`, `SameSite=None`, con `path=/api/auth` (ADR-012, ver Listado [lst:jwt-cookie]) – nunca en el cuerpo JSON, para que JavaScript del frontend no pueda leerlo ni reenviarlo manualmente:

```

private ResponseCookie buildRefreshCookie(String valor, long maxAgeMs) {
    return ResponseCookie.from(REFRESH_COOKIE_NAME, valor)
        .httpOnly(true)
        .secure(true)
        .sameSite("None")
        .path("/api/auth")
        .maxAge(Duration.ofMillis(maxAgeMs))
        .build();
}

```

El mismo método se reutiliza para *limpiar* la cookie en `logout` (`buildRefreshCookie("", 0)`, valor vacío y `maxAge=0`). El `accessToken`, de vida corta (1h), sigue viajando en el cuerpo JSON de la respuesta – migrarlo también a cookie está deliberadamente diferido (ver la nota de honestidad de ADR-012 y REQ-NF-002, 9) por el impacto que tendría en `jwt.interceptor.ts/ auth.service.ts` del frontend, no por un olvido.

Evaluación empírica y resultados

Este capítulo reporta, bloque por bloque, la evidencia empírica reunida para responder las cuatro preguntas de investigación del 1. Cada bloque sigue el mismo formato: pregunta de investigación asociada, métrica reportada, datos crudos con su ubicación exacta en el repositorio, estadística descriptiva real (no estimada) y una interpretación en prosa. **No todos los bloques están completos.** Dos (rendimiento, cobertura) tienen evidencia cerrada; uno (seguridad) está mayoritariamente cerrado con un componente diferido explícitamente a la Entrega Final; uno (calidad web) tiene evidencia real pero incompleta frente a lo que exige la guía; y uno (usabilidad) no se ha ejecutado todavía. Cada sección declara su estado real sin ajustar el texto para que suene más completo de lo que es – el mismo criterio que ya aplica el 12.

Bloque 1 – Rendimiento (k6)

Pregunta de investigación. RQ1 – ¿en qué medida el uso de caché (Redis) reduce la latencia observable del endpoint de catálogo bajo carga concurrente controlada, y con qué margen se cumplen los umbrales de rendimiento exigidos?

Métrica. Latencia `http_req_duration` (ms) de `GET /api/v1/libros` bajo 50 VUs concurrentes, en dos escenarios (`cache_caliente`, `cache_frio`), agregada sobre 5 corridas independientes.

Datos crudos. `docs/mediciones/perf/k6-run1.json` a `k6-run5.json` (formato NDJSON de k6, un `Point` por métrica por petición), analizados con `scripts/perf-analysis.py`. Detalle completo del protocolo (comando exacto, entorno, semillas) en el 4.

Estadística descriptiva (5 corridas agregadas).

Table 2: Estadística descriptiva de `http_req_duration` (ms) por escenario, 5 corridas agregadas

Escenario	<i>n</i>	Media	Mediana	σ	IC95% media	p90	p95	p99
cache_caliente	9 929	18,89	5,49	150,74	[15,93; 21,86]	11,66	19,49	104,79
cache_frio	10 074	4,84	4,34	2,88	[4,78; 4,89]	6,70	7,50	12,32

Ambos umbrales exigidos se cumplen con margen amplio: p95 19,49 ms < 200 ms (caliente), p95 7,50 ms < 500 ms (frío) (ver 7.1). Tasa de error HTTP ≥ 500 : 0,00 % (0 de 20 003 peticiones totales).

Comparación estadística pareada. Prueba de Wilcoxon de rangos con signo sobre el p95 de cada una de las 5 corridas (no sobre el *pool* agregado): estadístico = 0,0000, $p = 0,0625$ – no alcanza el umbral convencional de significancia ($p < 0,05$), por el límite de potencia estadística de $n = 5$ pares con dirección perfectamente consistente (`cache_caliente` > `cache_frio` en las 5 corridas, sin excepción; el p -valor exacto de dos colas mínimo alcanzable con $n = 5$ es exactamente 0,0625). Tamaño de efecto de Cliff’s delta = $-0,68$ (efecto **grande**, convención $|\delta| \geq 0,474$).

Correcciones por comparaciones múltiples. No aplican en este estudio: se realizó una única comparación pareada planificada (`cache_caliente` vs. `cache_frio`), no una familia de contrastes post-hoc simultáneos sobre los mismos datos; por tanto no corresponde ajustar el nivel de significancia con Bonferroni, Holm ni Benjamini-Hochberg.

Interpretación. La 7.1 muestra un efecto grande y consistente en las 5 corridas – `cache_caliente` es sistemáticamente más lento que `cache_frio`, un resultado que a primera vista es contraintuitivo (se esperaría que el escenario *con* caché fuera el más rápido). La explicación no es que el caché perjudique el rendimiento: es una limitación metodológica ya documentada del escenario `cache_frio`, que –por construcción del PRNG determinista y el tamaño reducido de la tabla `libro` en la seed de desarrollo– termina midiendo sobre todo consultas con página vacía fuera de rango, no el costo real de traer una página llena sin caché (`docs/mediciones/perf/REPORT.md`, punto 3 de su “Análisis breve”; también citado en `docs/arquitectura/ISO25010.md`). Esta limitación se retoma con detalle en el 12 (validez de constructo y de conclusión) y es la razón por la que este capítulo no interpreta el resultado como “el caché no ayuda” – ambos umbrales de la guía se cumplen igual, con margen amplio, y esa es la respuesta directa a RQ1 dentro del alcance que esta comparación sí puede sostener.

Bloque 2 – Seguridad (OWASP Top 10:2021)

Pregunta de investigación. RQ2 – ¿qué proporción de los controles auditados del OWASP Top 10:2021 presenta hallazgos reales en la implementación, y en qué proporción de esos hallazgos el propio equipo aplicó una corrección verificable dentro del mismo ciclo de desarrollo?

Métrica. Estado por control (hallazgo real: sí/no; corrección verificada: sí/no/diferida), sobre 6 controles evaluados contra el stack Docker real. **Nota metodológica:** esta métrica es categórica, no continua – no aplica una estadística de media/desviación típica/IC 95% en el sentido del Bloque 1; la estadística descriptiva pertinente aquí es la proporción simple sobre 6 controles, reportada abajo.

Datos crudos y resultado por control.

Table 3: Estado real por control

Control	Estado real
A01 – Control de acceso roto	Hallazgo real encontrado y corregido (rol admin en libros)
A02 – Fallos criptográficos / TLS	Decisión de arquitectura, preparación del backend y verificación de despliegue real CERRADAS (ADR-015, for
A03 – Inyección	Verificado sin hallazgos (inyección SQL)
A05 – Mala configuración de seguridad	Cerrado con verificación real (CSP, no-root, Swagger 404 en prod); 1 hallazgo real encontrado y corregido en el
A07 – Fallos de identificación y autenticación	Hallazgo real encontrado y corregido (rate limiting de login ausente)
A09 – Fallos de registro y monitoreo	Hallazgo real encontrado y corregido (logging de autenticación insuficiente)

Re-verificación automatizada. Los 4 controles con hallazgo corregido y verificable por `curl` (A01, A03, A07, A09) se re-verificaron con `scripts/owasp-audit.sh` contra el flujo real de verificación de correo obligatoria agregado después del hallazgo original, y los 4 vuelven a pasar (`docs/mediciones/sec/2026-08-11-owasp-audit-automatizado.md`).

Estadística descriptiva (proporciones sobre 6 controles). Hallazgo real encontrado: **4 de 6** (A01, A05, A07, A09), = 66,7 %. De esos 4, corrección verificada dentro del mismo ciclo: **4 de 4**, = 100 %. Sin hallazgo real de vulnerabilidad, cerrados con verificación completa: **2 de 6** (A03, verificado sin hallazgo desde la corrida original; A02, decisión de arquitectura más verificación real de despliegue con el header `Strict-Transport-Security` confirmado en producción el 18-ago-2026). Con esto, **6 de 6 controles** (= 100 %) quedan con resultado cerrado – ninguno permanece parcial o diferido.

ZAP – baseline y Ajax Spider (ejecutados). Dos corridas de OWASP ZAP quedaron ejecutadas y versionadas después de escribirse el párrafo original de este bloque – por eso decía “aún no ejecutado”; ese estado ya no es real y se corrige aquí –: un *baseline scan* (docs/mediciones/sec/zap/2026-08-16-zap-baseline-report, en JSON/HTML/MD) y un *Ajax Spider scan* más exhaustivo (docs/mediciones/sec/zap/2026-08-17-zap-ajax-full-report, en JSON/HTML/MD). Ambas corridas apuntan al stack Docker **local** (host.docker.internal, puertos 4200 y 14200 respectivamente), no al despliegue público de producción – una corrida anterior y más limitada sí apuntó a producción (docs/mediciones/sec/zap/reporte-zap-baseline-frontend-prod-2026-08-14-v3.xml, contra <https://biblora-sgb.onrender.com>) y no reportó hallazgos *High* (0 High, 1 Medium, 1 Low), pero es una corrida más antigua y menos exhaustiva que las dos de esta sección, y no sustituye su cobertura.

Estadística descriptiva (ambas corridas, local). *Baseline* (2026-08-16): 1 *High*, 1 *Medium*, 5 *Low*, 2 *Informational*. *Ajax Spider* (2026-08-17): 2 *High*, 2 *Medium*, 5 *Low*, 2 *Informational*.

Los 2 hallazgos *High* del Ajax Spider, explicados con honestidad, no ocultados ni retirados del reporte. Los archivos versionados en docs/mediciones/sec/zap/ no se editan – son el registro real de lo que esa corrida encontró en ese momento –, pero ninguno de los dos refleja hoy una vulnerabilidad activa en el código propio del proyecto:

- **DOM-based XSS (taint flow)**, 8 instancias. Se verificó con `grep` sobre `frontend-angular/src/app/` que el código propio de la aplicación tiene **cero** usos de `insertBefore`, `.innerHTML` o `bypassSecurityTrust*` – las tres vías reales de XSS DOM-based en una aplicación Angular. El flujo de contaminación que ZAP detecta vive dentro del motor de renderizado interno de Angular, empaquetado en el *bundle* minificado (`main-*.js`), no en código de este proyecto; el análisis estático de ZAP no distingue entre ambos orígenes.
- **Vulnerable JS Library**, 1 instancia. La evidencia reportada es literalmente `["ng-version", "17.3.12"]` – pero `frontend-angular/package.json` especifica hoy `@angular/core` en la versión 21.2.20. La corrida escaneó un *build* de ANTES del *upgrade* de Angular hecho hoy mismo (17.3 → 21.2.20, específicamente para cerrar los CVE conocidos de Angular hasta la 18.2.14); es un hallazgo desactualizado por construcción del propio experimento, no una vulnerabilidad vigente. Repetir la corrida contra el *build* ya actualizado habría sido lo correcto para confirmarlo, pero no hubo tiempo antes del cierre de esta entrega – se declara esa limitación en vez de asumir el resultado sin verificarlo.

Interpretación. La respuesta directa a RQ2, con los números reales de arriba: de 6 controles auditados, 4 (66,7 %) presentaron un hallazgo real, y de esos 4, el 100 % recibió una corrección verificable dentro del mismo ciclo de desarrollo, re-confirmada después contra un flujo de autenticación que cambió (verificación de correo obligatoria). La auditoría ZAP automatizada, distinta de la verificación manual control por control, sí quedó ejecutada dos veces contra el entorno local – no aporta hallazgos *High* vigentes contra el código propio del proyecto, aunque sí dos hallazgos reales que quedan explicados arriba en vez de ocultados, y una re-corrida post-*upgrade* de Angular queda como trabajo pendiente explícito.

Sobre A02, cerrado en su totalidad: la decisión de arquitectura y la preparación del backend están documentadas (ADR-015, `forward-headers-strategy`), y la verificación de despliegue real que antes faltaba ya se completó – el header `Strict-Transport-Security` se verificó activo en producción (`curl -sI https://biblora-sgb.onrender.com/`, 18-ago-2026, no simulado ni asumido): `strict-transport-security: max-age=315360000; includeSubdomains; preload`. Con el header real confirmado, A02 no queda solo preparado arquitectónicamente: queda resuelto por completo (arquitectura, backend y despliegue verificado), el sexto y último de los 6 controles auditados en cerrar.

Los tres reportes ZAP completos (JSON/HTML/MD, incluida la corrida más antigua contra producción) quedan versionados sin editar en docs/mediciones/sec/zap/ como registro crudo de cada corrida.

Bloque 3 – Cobertura de pruebas automatizadas (JaCoCo)

Pregunta de investigación. RQ4 (componente de cobertura) – qué proporción de la base de código cuenta con prueba automatizada real, como parte de evaluar el efecto de la estrategia híbrida de acceso a datos sobre la cobertura de pruebas.

Métrica. Porcentaje de líneas, ramas y complejidad ciclomática cubiertas, medido con `jacoco-maven-plugin`. La cifra de cierre vigente se re-verificó con una corrida real de `./mvnw verify` sobre HEAD 568e8c1f (rama `Presentacion_Final`): 464 tests, 0 fallos, 2 *skipped*, **56,30 %** de líneas (2195/3899). Esta cifra supera a la vigente anterior (56,01 %, HEAD 89eb6ab, documentada sin modificar en 2026-08-27-cobertura-jacoco-cierre-final.md) por los tests de capa `controller` agregados el 6 de septiembre de 2026, y reemplaza también a la cifra aún más antigua (82,97 % en commit 0d1474d) que no tenía artefacto versionado asociado.

Retractación de la cifra 61,50 %. Una auditoría de rúbrica detectó que la cifra citada previamente en este capítulo para el commit 825ad34 (61,50 %, 1481/2408) no coincidía con el artefacto XML ya versionado en

`docs/mediciones/jacoco/2026-08-25-jacoco-final/` (55,25 %, 1541/2789). Se reconstruyó 825ad34 de forma aislada (*git worktree*) **dos veces de forma independiente**, y ambas reconstrucciones arrojan 55,25 %, nunca 61,50 %. La cifra de 61,50 % se retracta aquí explícitamente como **error de origen en el texto narrado** – no un artefacto correcto que se perdió o fue sobrescrito, como se conjeturó al detectar la discrepancia. El artefacto XML de 825ad34 en sí es y siempre fue correcto; no se modifica. La base de líneas analizables creció de forma legítima de 2789 a 2880 entre 825ad34 y HEAD por la incorporación de funcionalidad real del módulo de auditoría (no por el error de cifra ahora corregido).

Datos crudos. `docs/mediciones/jacoco/report.xml`, `report.csv`, `html/index.html` (reporte navegable completo) corresponden ahora a la corrida vigente sobre HEAD 568e8c1f descrita arriba – reemplazan en el mismo lugar al artefacto de a2c88f8 (13-ago) que ocupaba antes esa ruta; narrativa y desglose por clase de aquella medición de 13-ago quedan sin modificar en `docs/mediciones/jacoco/2026-08-13-cobertura-jacoco-post-merge-8-modulos.md`. La medición de 825ad34 (55,25 %, re-verificada, correcta) se documenta en `2026-08-25-cobertura-jacoco-final.md`, con sus artefactos XML/HTML/CSV en `docs/mediciones/jacoco/2026-08-25-jacoco-final/` (archivo y artefacto sin modificar). La medición previa de cierre (HEAD 89eb6ab, 56,01 %) se documenta en `2026-08-27-cobertura-jacoco-cierre-final.md` con sus artefactos en `docs/mediciones/jacoco/2026-08-27-jacoco-cierre-final/` (archivo y artefacto sin modificar). La medición intermedia previa (c2d76ad, 61,30 %) permanece documentada sin modificar en `2026-08-25-cobertura-jacoco-cierre-entrega-final.md` como registro honesto del proceso.

Estadística descriptiva. Esta métrica es una medición agregada sobre la totalidad del código analizado (95 clases en la corrida vigente, tras exclusiones documentadas de DTOs/entidades/configuración de framework), no una muestra de un proceso aleatorio – no aplica media/desviación típica/IC 95% de la misma forma que el Bloque 1; el número reportado es el porcentaje real exacto, sin margen de muestreo.

Table 4: Cobertura JaCoCo: post-merge de 8 módulos de dominio, cifra de 825ad34 re-verificada (retracta el 61,50 % citado anteriormente para ese commit), cierre del 27-ago y corrida vigente al cierre de la Entrega Final (`Presentacion_Final`, HEAD 568e8c1f). La columna 30-jul (pre-merge) del historial original se omite aquí por espacio de tabla; permanece documentada sin modificar en `2026-07-30-cobertura-jacoco-dominio-servicios.md`.

Métrica	13-ago (post-merge, a2c88f8)	825ad34 (re-verificado)	27-ago (HEAD 89eb6ab)	Vigente (cierre, HEAD 568e8c1f)	Umbral
Lines	81,64 % (1014/1242)	55,25 % (1541/2789)	56,01 % (1613/2880)	56,30 % (2195/3899)	≥70 % No
Branches	58,05 % (137/236)	34,24 % (239/698)	34,59 % (247/714)	30,83 % (390/1265)	–
Complexity	65,16 % (288/442)	45,47 % (427/939)	45,70 % (436/954)	42,35 % (620/1464)	–

Interpretación. El resultado responde a RQ4 con un dato concreto y un artefacto versionado que lo respalda: la base de líneas analizables creció de 1242 (13-ago) a 3899 (cierre) con los commits posteriores al merge de los 8 módulos (nuevos controladores, chatbot, auditoría, QR, backups, sugerencias de adquisición, etc.), y la cobertura global de líneas se mantiene **por debajo del umbral informal de 70 %** fijado por el equipo para la Entrega Final (56,30 % al cierre). El paquete más bajo sigue siendo *integration* (12,1 %, clase `GeminiClient`), seguido de *chatbot* (13,6 %), *chatbot.tool* (38,4 %) y *exception* (38,6 %). Sin embargo, a diferencia de la medición anterior, el criterio real de la rúbrica (P1: ≥65 % en al menos 2 de 3 capas *controller/service/security*) **sí se cumple al cierre: controller** subió a 97,1 % (564/581) y *security* se mantiene en 85,2 % (109/128), 2 de las 3 capas exigidas; *service* queda como brecha conocida en 51,3 % (1329/2591), y *scheduling* (98,3 %) sigue por encima individualmente aunque no forma parte de las 3 capas del criterio P1. Este resultado contradice la cifra de 82,97 % citada en una versión anterior de este documento (commit 0d1474d) que carecía de artefacto versionado asociado; se corrigió aquí con honestidad y se versionó el reporte real correspondiente al commit 825ad34. La medición intermedia en c2d76ad (61,30 %) se mantiene documentada en `2026-08-25-cobertura-jacoco-cierre-entrega-final.md` como paso previo, sin modificar.

La columna “825ad34 (re-verificado)” corrige, no repite, lo que este capítulo citaba antes. La cifra anterior para ese commit (61,50 %, 1481/2408) no sobrevivió una re-verificación independiente: se reconstruyó 825ad34 dos veces en un *worktree* aislado y ambas veces se obtuvo 55,25 % (1541/2789), igual que el artefacto ya versionado en `docs/mediciones/jacoco/2026-08-25-jacoco-final/jacoco.xml`. Se retracta 61,50 % como error de origen en el texto narrado, siguiendo la práctica ya establecida en este informe de documentar correcciones de medición en vez de editarlas en silencio. La columna vigente (HEAD 568e8c1f, 56,30 %) es la que corresponde citar como cifra actual de cierre; 825ad34 y la medición previa del 27-ago (HEAD 89eb6ab, 56,01 %) quedan como puntos de referencia histórica correctos, sin modificar.

Table 5: Cobertura JaCoCo por capa arquitectónica (commit 825ad34)

Capa	Líneas cubiertas/total	Porcentaje	≥65 %?
Controller	194/311	62,38 %	No

Capa	Líneas cubiertas/total	Porcentaje	≥65 %?
Service	1086/1740	62,41 %	No
Domain/Entity	—	Excluido	N/A

Interpretación por capa (7.4, commit 825ad34, histórico). Ni **Controller** (62,38 %) ni **Service** (62,41 %) alcanzan individualmente el umbral del 65 % exigido por el criterio P1 de la guía para clasificar la cobertura como “Satisfactorio” (requiere al menos 2 de 3 capas ≥65 %). Por tanto, la clasificación de cobertura por capa cae en la categoría “En desarrollo” (50–65 %), consistente con la cifra global re-verificada de 55,25 % para este mismo commit (825ad34) reportada en 7.3 – no con el 61,50 % citado antes de la retractación. Esta tabla e interpretación quedan sin modificar como referencia histórica; la tabla siguiente documenta la medición vigente al cierre.

Table 6: Cobertura JaCoCo por capa arquitectónica, medición vigente al cierre (Presentacion_Final, HEAD 568e8c1f)

Capa	Líneas cubiertas/total	Porcentaje	≥65 %?
Controller	564/581	97,07 %	Sí
Security	109/128	85,16 %	Sí
Service	1329/2591	51,29 %	No
Domain/Entity	—	Excluido	N/A

Interpretación por capa (vigente al cierre, 7.5). Los tests de controlador agregados el 6 de septiembre de 2026 cambian el resultado de este criterio: **Controller** pasó de 62,38 % a 97,07 %, y junto con **Security** (85,16 %, sin cambios respecto a mediciones previas) se alcanzan 2 de las 3 capas exigidas por el criterio P1 (≥65 % en al menos 2 de 3), por lo que la clasificación de cobertura por capa pasa de “En desarrollo” a “**Satisfactorio**”. **Service** permanece como brecha conocida (51,29 %, prácticamente sin cambio respecto al histórico), y se documenta aquí sin corregirse en esta entrega – ver 7.3 para el detalle completo de artefactos.

La capa **Domain/Entity** (`com.uteq.backend.entity`) está deliberadamente excluida de la medición JaCoCo, tal como se declara en `backend-springboot/pom.xml` mediante las exclusiones: `com/uteq/backend/entity/**` (entidades JPA con getters/setters generados por Lombok, sin lógica de negocio propia), `com/uteq/backend/dto/**` (DTOs de transporte, solo estructura de datos), `com/uteq/backend/config/**` (configuración de beans/framework, probada indirectamente por tests de integración), y `com/uteq/backend/BackendApplication.class` (punto de entrada, sin lógica testable aislada). Estas exclusiones reflejan que dichos paquetes no aportan lógica de negocio medible en términos de cobertura de línea/rama; el código testable con lógica de negocio real reside en **service**, **controller** y **security**, que sí son medidos. Con **controller** y **security** ya por encima del 65 % al cierre, el equipo identifica el aumento de cobertura en **service** (51,29 %) como la brecha prioritaria pendiente, manteniendo la exclusión de entidades/DTOs/config como decisión de diseño documentada y razonada.

Bloque 4 – Calidad web (Lighthouse)

Pregunta de investigación. Ninguna de las cuatro RQ del 1 cubre explícitamente calidad web/Lighthouse – se declara esto con honestidad en vez de forzar una relación artificial con RQ1–RQ4; este bloque documenta un instrumento exigido por la guía (Bloque C.5) sin una pregunta de investigación propia todavía formulada para él.

Métrica. Scores 0–100 de Lighthouse en las categorías Performance, Accessibility, Best Practices y SEO, perfil móvil con *throttling* Slow 4G.

Datos crudos. `docs/mediciones/lighthouse/lhci-20260731-0300.json` (corrida “antes”), `lhci-20260731-0330.json` (corrida “después”, tras 2 correcciones de SEO, ambas contra `localhost`); 6 corridas reales contra producción (<https://biblora-sgb.onrender.com/>) que expusieron un defecto real de *layout shift*: `lhci-mobile-prod-20260817-*.json` (3), `lhci-desktop-prod-20260817-*.json` (3); y 6 corridas reales de re-confirmación contra producción, ejecutadas después de corregir ese defecto y verificar el redeploy (commit `fced67d`): `lhci-mobile-prod-20260818-*.json` (3), `lhci-desktop-prod-20260818-*.json` (3) – **12 corridas de producción en total**, sumadas a las 2 originales contra `localhost`. Narrativa completa, con comandos ejecutados y cada paso de verificación, en `docs/mediciones/lighthouse/REPORT.md`.

Table 7: Scores Lighthouse reales por corrida. Las 2 primeras filas son la muestra histórica contra `localhost`; las 4 filas siguientes agregan media y desviación típica de las 12 corridas reales de producción (2 rondas × 2 perfiles × 3 corridas)

Perfil	Corrida	Performance	Accessibility	Best Practices	SEO
Móvil	<code>lhci-20260731-0300 (localhost)</code>	95	95	100	82
Móvil	<code>lhci-20260731-0330 (localhost)</code>	99	100	100	100

Perfil	Corrida	Performance	Accessibility	Best Practices	SEO
Móvil	prod. 17-ago, antes del fix (media, 3)	86,0 (DT 13,0)	100,0 (DT 0,0)	100,0 (DT 0,0)	100,0 (DT 0,0)
Escritorio	prod. 17-ago, antes del fix (media, 3)	88,0 (DT 0,0)	100,0 (DT 0,0)	100,0 (DT 0,0)	100,0 (DT 0,0)
Móvil	prod. 18-ago, después del fix (media, 3)	86,3 (DT 1,5)	100,0 (DT 0,0)	100,0 (DT 0,0)	100,0 (DT 0,0)
Escritorio	prod. 18-ago, después del fix (media, 3)	100,0 (DT 0,0)	100,0 (DT 0,0)	100,0 (DT 0,0)	100,0 (DT 0,0)
Umbral exigido		≥80	≥90	≥90	≥90

Estadística descriptiva (7.6). Con 3 corridas por perfil en cada una de las dos rondas de producción (12 corridas en total, sumadas a las 2 originales contra `localhost`), ya hay muestra suficiente para reportar media y desviación típica reales por categoría y perfil – calculadas directo de los *scores* crudos de cada JSON, sin estimar. Esto reemplaza la limitación de muestra ($n = 2$, un solo perfil) declarada en una versión anterior de este capítulo.

Cobertura de corridas – resuelta. Existen **12 corridas reales contra producción**: 3 móvil + 3 escritorio en la ronda del 17-ago (que expuso el defecto de CLS descrito abajo) y 3 móvil + 3 escritorio en la ronda del 18-ago, después de corregirlo y verificar el redeploy – cumpliendo con margen el requisito de la guía de “al menos 3 corridas por perfil (móvil, escritorio)”. Las 2 corridas originales contra `localhost` se conservan como evidencia histórica del hallazgo y corrección de SEO, no se descartan.

Hallazgo real – defecto de *Cumulative Layout Shift* encontrado, diagnosticado y corregido con evidencia de producción. Las 6 corridas de la ronda del 17-ago expusieron un defecto real: **CLS de 0,235–0,236, consistente en las 3 corridas de escritorio** (franja “pobre” de Lighthouse, $> 0,25$), y un pico intermitente de **0,338 en 1 de 3 corridas móviles** (las otras 2 en 0,013). Se investigó la causa raíz con el *audit layout-shifts* del propio JSON (no se adivinó): el elemento responsable del 99,9 % del CLS es `main.max-w-container-max` de `PortalPublicoComponent` – no el panel CTA que se había señalado como sospechoso en una nota preliminar de `REPORT.md`, corrección que se documenta ahí explícitamente en vez de dejarse como si hubiera sido correcta. La causa concreta, confirmada en el código fuente: el grid del catálogo público mostraba una sola línea de texto (“Cargando catálogo...”) mientras cargaba, reemplazada de golpe por un grid completo de 10 tarjetas al llegar los datos, sin espacio reservado. Se corrigió con un estado de carga tipo *skeleton* (10 tarjetas placeholder con la estructura real) en `portal-publico.component.html/.ts` (commit `fc6d67d`).

Verificado primero en local (CLS de 0,236 a 0,032 de media, 86 % de reducción, *stack* completo con backend real, 141/141 tests sin regresión) y luego **re-confirmado contra producción real** tras el redeploy – verificado comparando el hash del *bundle* servido (`main-CCW77Z7S.js` → `main-KA6MN67F.js`) antes de medir nada, sin asumir el redeploy por un aviso informal de que “ya estaba arreglado”. Las 6 corridas de re-confirmación (18-ago) dan:

Table 8: Cumulative Layout Shift antes y después del fix, 12 corridas reales de producción. *Móvil “antes” no se resume en media/DT: 2 corridas en 0,013 y 1 anómala en 0,338 (ver `REPORT.md` para el detalle de esa corrida puntual)

Perfil	CLS antes (producción, 17-ago)	CLS después (producción, 18-ago)
Escritorio	0,236 / 0,236 / 0,235 (media 0,236, DT 0,000)	0,031 / 0,031 / 0,031 (media 0,031, DT 0,0003)
Móvil	0,013 / 0,013 / 0,338*	0,005 / 0,011 / 0,011 (media 0,009, DT 0,003)

Como resume 7.7, Escritorio pasa de la franja “pobre” ($> 0,25$) a “buena” ($< 0,1$), una reducción del 87%; el pico anómalo de móvil no se repite en ninguna de las 3 corridas de re-confirmación. *Score* de *audit* de CLS: 1,0 (perfecto) en las 6 corridas post-fix. Los números de producción coinciden casi exactamente con la verificación local (0,031 producción vs. 0,031–0,033 local), confirmando que la verificación local había sido representativa y no un artefacto del entorno de prueba.

Interpretación. Con las 12 corridas de producción, las 4 categorías de Lighthouse cumplen su umbral en las 6 corridas individuales de la ronda post-fix (no solo en la media), y CLS queda confirmado en rango “bueno” en producción real, no solo en verificación local – este es un hallazgo empírico completo: un defecto real detectado por la propia medición (no buscado a propósito), con causa raíz confirmada en código fuente, corrección aplicada y verificación antes/después en producción, no solo localmente. Una nota de honestidad ya documentada en `REPORT.md` sobre la muestra histórica contra `localhost`, que este capítulo preserva: Accessibility subió de 95 a 100 y Performance de 95 a 99 entre esas 2 corridas **sin que se tocara ningún estilo, color ni optimización de rendimiento** – la explicación más probable es variabilidad normal entre corridas de Lighthouse, no un efecto causal de las correcciones de SEO aplicadas en ese momento; no se le atribuye esa mejora a los cambios.

Limitación restante. La anomalía puntual de Performance móvil en la ronda “antes” (Run 3, $71 < 80$, con Speed Index de 48,1 s y TTFB $10\times$ más alto que las otras 2 corridas de esa ronda) no se repite en ninguna de las 3 corridas post-fix – se documenta como variabilidad de infraestructura compartida del *hosting* en el momento de esa captura puntual, no

como un defecto de aplicación reproducible, y no se le atribuye causalmente al fix de CLS (son métricas distintas). No se investiga más a fondo por alcance de esta entrega.

Table 9: Resumen Lighthouse móvil vs escritorio: media $\pm$ DT sobre 3 corridas de producción por perfil (commit 825ad34). Umbrales: Performance ≥ 80 , Accessibility ≥ 90 , Best Practices ≥ 90 , SEO ≥ 90 .

Categoría	Escritorio (media $\pm$ DT)	Móvil (media $\pm$ DT)	Umbral
Performance	100,0 $\pm$ 0,0	86,3 $\pm$ 1,5	≥ 80
Accessibility	100,0 $\pm$ 0,0	100,0 $\pm$ 0,0	≥ 90
Best Practices	100,0 $\pm$ 0,0	100,0 $\pm$ 0,0	≥ 90
SEO	100,0 $\pm$ 0,0	100,0 $\pm$ 0,0	≥ 90

Como resume 7.8, las 4 categorías cumplen su umbral en **ambos perfiles** (móvil y escritorio), en las 6 corridas individuales (3 por perfil). Escritorio alcanza 100,0 en todas las categorías (DT 0,0); móvil alcanza 100,0 en Accessibility, Best Practices y SEO, y 86,3 $\pm$ 1,5 en Performance (margen cómodo sobre 80). Se cumple con margen el requisito de la Guía: “al menos tres corridas por perfil (mobile, desktop) con media y desviación típica por categoría” (P4/B.10).

Bloque 5 – Usabilidad (SUS)

Pregunta de investigación. RQ3 – ¿cómo perciben usuarios reales la usabilidad del sistema, medida con el instrumento estandarizado System Usability Scale (SUS)?

Métrica. Puntuación SUS agregada (0–100), instrumento de 10 ítems (?) (4).

Datos crudos. Ninguno todavía: $N = 0$ de los $N \geq 15$ participantes que exige la guía. El archivo docs/mediciones/sus/sus.csv se conserva únicamente como **datos de prueba (mock/sintéticos) para validación del pipeline de análisis automatizado**, sin valor evidencial: una corrida declarada anteriormente como $N = 15$ se retiró por falta de trazabilidad a un export crudo del instrumento y patrones incompatibles con respuestas independientes (ver OBS-08 en docs/observaciones/OBSERVACIONES.md). El pipeline (scripts/sus-analysis.ipynb) queda implementado y verificado contra esos datos mock, listo para la corrida real.

Estadística descriptiva. Sin datos no hay estadística que reportar: la tabla, el intervalo de confianza y la clasificación Bangor de la versión anterior de este bloque quedan retirados junto con la corrida. Se conserva el método para la corrida futura: puntuación SUS agregada 0–100 por participante con la fórmula de Brooke, media, mediana, DT, IC 95 % con t de Student ($df = N - 1$, dado $N < 30$) y clasificación en la escala de adjetivos de Bangor frente al umbral de aceptabilidad de 68 puntos; desglose por ítem Q1–Q10 en escala Likert 1–5 más los ítems suplementarios I1/I2 del instrumento local.

Protocolo (permanente, ya versionado). Plantilla de consentimiento informado, muestreo por conveniencia y anonimización por código de participante (docs/etica/consentimientos/plantilla.md, 4).

Estado real. Pendiente. La toma de datos con usuarios y el protocolo SUS se difieren formalmente como trabajo futuro para la fase de despliegue en producción (10.1). Documentado como reabierto en OBS-08 de docs/observaciones/OBSERVACIONES.md. Las 7.2 y 7.3 muestran la salida del pipeline sobre esos datos mock (sin valor evidencial).

Resumen de bloques

Table 10: Estado real de cada bloque de eva

Bloque	Estado
1. Rendimiento (k6)	5/5 corridas, umbrales cumplidos, limitación de constructo declarada
2. Seguridad (OWASP)	6/6 controles con resultado cerrado (A02 incluido, HSTS verificado en producción); ZAP ejecutado (0 High en producción, 2 High en desarrollo)
3. Cobertura (JaCoCo)	Medición vigente post-merge: criterio P1 de la rúbrica cumplido (controller 97,1 %, security 85,2 %, 2 de 3 capas ≥ 65 %); service 100 %
4. Calidad web (Lighthouse)	12 corridas reales de producción (2 rondas $\times$ 2 perfiles $\times$ 3), umbral cumplido en las 6 corridas post-fix; defecto real de CLS encontrado
5. Usabilidad (SUS)	$N = 0$ de $N \geq 15$ exigidos; protocolo y pipeline listos, corrida diferida a despliegue

De los cinco bloques exigidos por la guía, cuatro están completos con umbral cumplido (rendimiento, seguridad, cobertura, calidad web) y uno (usabilidad) queda pendiente con $N = 0$. Esta distribución de “4 de 5 completos” es el estado real del proyecto al momento de escribir este capítulo: la corrida SUS declarada anteriormente se retiró por falta de trazabilidad (ver OBS-08 en docs/observaciones/OBSERVACIONES.md) y la toma de datos con usuarios se difiere formalmente a la fase de despliegue en producción.

Discusión

Este capítulo interpreta la evidencia del 7 a la luz de las cuatro preguntas de investigación planteadas en el 1 – sin repetir las cifras ya reportadas allí, sino sintetizándolas y discutiendo lo que significan. Se cierra comparando SGB-SaaS contra la literatura revisada en el 2, señalando los hallazgos no buscados que surgieron durante el desarrollo, y discutiendo las implicaciones prácticas del proyecto para una biblioteca real.

Respuesta a las preguntas de investigación

Retomando las cuatro preguntas planteadas en la 1.2:

RQ1 – ¿En qué medida el caché reduce la latencia, y con qué margen se cumplen los umbrales?

Ambos umbrales de rendimiento se cumplen con margen amplio – p95 19,49 ms contra un límite de 200 ms en caché caliente, p95 7,50 ms contra 500 ms en frío (7.1)–, así que en el sentido estricto que exige la guía, la respuesta a RQ1 es afirmativa: el sistema cumple. Pero la comparación *entre* escenarios no permite concluir de forma limpia que sea el caché específicamente el responsable de la diferencia observada, porque el escenario `cache_frio` tiene una limitación metodológica real – termina midiendo mayoritariamente consultas con página vacía, no la consulta completa que `cache_frio` debería representar (7.1, 12). Lo que sí se sostiene pese a esa limitación es la dirección y el tamaño del efecto entre corridas de `cache_caliente`: Cliff's delta = $-0,68$ (grande, consistente en las 5 corridas), exactamente la dirección que la hipótesis de caché predeciría si la comparación fuera limpia. En síntesis: RQ1 se responde con evidencia de cumplimiento de umbral sólida, y con evidencia de efecto del caché sugestiva pero no aislada limpiamente – una distinción que este documento no difumina.

RQ2 – ¿Qué proporción de controles OWASP presentó hallazgos reales, y cuántos se corrigieron?

De los 6 controles auditados, 4 (A01, A05, A07, A09) presentaron un hallazgo real – el 66,7% reportado en 7.2 – y los 4 recibieron corrección verificable dentro del mismo ciclo de desarrollo, re-confirmada después contra un flujo de autenticación que cambió. Este es, en sí mismo, el dato más relevante para RQ2: la mayoría de los controles auditados **sí** tenían defectos reales en la implementación – rol admin mal validado, rate limiting de login ausente, logging de autenticación insuficiente, y un código de estado HTTP incorrecto detectado durante la propia verificación de A05 (500 en vez de 404 en Swagger bajo perfil `prod`). No fue una auditoría de casillas vacías que confirma lo que ya se sabía: encontró y forzó a corregir defectos de seguridad concretos. El control A03 (inyección) se verificó limpio, sin hallazgo – un resultado real también, no la ausencia de una prueba. A02 también queda cerrado: la decisión de arquitectura y la preparación del backend para TLS ya estaban cerradas, y la verificación final que dependía de una pieza de infraestructura externa al propio código – un despliegue público real, no trabajo de ingeniería adicional – se completó el 18-ago-2026 contra el despliegue real de producción (<https://biblora-sgb.onrender.com/>): el header `Strict-Transport-Security` está activo (`max-age=315360000; includeSubdomains; preload`). Con los 6 controles auditados cerrados, RQ2 se responde de forma completa, no solo con la proporción de hallazgos y su corrección, sino también con la verificación final del único control que dependía de un despliegue público.

RQ3 – ¿Cómo perciben usuarios reales la usabilidad del sistema (SUS)?

RQ3 **no tiene respuesta todavía**. Con $N = 0$ de los $N \geq 15$ participantes exigidos, no existe ninguna puntuación SUS que interpretar (7.5). Forzar una respuesta aquí – aunque fuera calificada de “preliminar” o “estimada” – sería exactamente el tipo de afirmación no verificada que este documento se niega a producir en cualquier otro capítulo; no hay razón para relajar ese criterio en la discusión. La respuesta honesta a RQ3, a la fecha de este documento, es que sigue siendo trabajo en progreso, bloqueado en la cadena de dependencias correcta (protocolo de consentimiento ya definido → despliegue público → reclutamiento → aplicación del instrumento), no una omisión arbitraria.

RQ4 – ¿Qué efecto tiene la estrategia híbrida CRUD-ORM/SP sobre trazabilidad y cobertura?

La matriz de trazabilidad (5.6, 5) muestra una distribución real y no pareja: 48,8% de los requisitos son CRUD-ORM exclusivo, 18,6% son procedimiento/función SQL exclusivo, y solo 4,7% son genuinamente híbridos (requisitos transversales que combinan ambos mecanismos) – el resto no aplica un patrón de acceso a datos directo. La cobertura de pruebas automatizadas, medida sobre esa misma base de código, es del 81,64% de líneas (7.3), muy por encima del umbral exigido, y no se concentra de forma distinta entre el código que usa ORM y el que usa SP – ambos paquetes (`service`, que contiene la mayoría de las invocaciones a SP, y el resto orientado a ORM) están sobre el 89%. Con esta evidencia, RQ4 tiene una respuesta afirmativa en cuanto a resultado: la estrategia híbrida no perjudicó ni la trazabilidad (cada requisito tiene su mecanismo de acceso documentado en la matriz) ni la cobertura de pruebas. Pero

el dato más honesto para RQ4 no es un número de cobertura – es que la estrategia híbrida generó **fricción real de implementación**: el equipo tuvo que abandonar `@Procedure/@NamedStoredProcedureQuery` (el mecanismo JPA formalmente diseñado para invocar procedimientos almacenados) y migrar a `@Query` nativa con parámetros bindeados, por un defecto documentado de la combinación Hibernate 6.2+/7.x con procedimientos multi-OUT en PostgreSQL (docs/mediciones/backend/2026-07-28-fallo-invocacion-sp-multi-out.md, spring-projects/spring-data-jpa#3393, ya detallado en el 6). Es decir: la arquitectura híbrida no fue gratis – costó un ciclo de depuración real y un cambio de mecanismo de invocación no planeado originalmente – y ese costo es tan parte de la respuesta a RQ4 como el resultado final positivo de cobertura y trazabilidad.

Comparación con trabajos relacionados

El 2 identificó una brecha concreta en la literatura revisada: ningún trabajo de los 10 sistemas primarios comparados combina, en un solo sistema, autenticación JWT con roles, catálogo con autocompletado de ISBN, un chatbot con IA generativa real usado tanto para recomendaciones como para consultas, y una arquitectura híbrida de acceso a datos con evidencia empírica de rendimiento reproducible. Con los resultados reales de este documento en mano, esa brecha se sostiene y se refuerza con evidencia, no solo con ausencia de contraejemplos: ? identifica el problema de rendimiento que motiva una arquitectura híbrida ORM/SP pero no llega a implementarla ni medirla; SGB-SaaS sí la implementa y la mide (RQ4, arriba) – incluyendo el costo real de implementarla, un dato que ese trabajo tampoco podría reportar por no haber construido el sistema. ? mide el efecto de Redis con métricas de uso de CPU y tráfico, pero sin la evaluación estadística formal (Wilcoxon pareado, Cliff’s delta) que este documento aporta para su propio caso de uso de caché (RQ1, arriba) – aunque, con honestidad, esa misma evaluación formal es la que expuso la limitación metodológica de `cache_frio` que un enfoque menos riguroso no habría descubierto. Ningún trabajo comparado en el 2 reporta una auditoría de seguridad control por control con hallazgos reales corregidos verificablemente (RQ2) ni una matriz de trazabilidad de 46 requisitos con patrón de acceso a datos declarado por requisito (RQ4) – estas dos piezas de evidencia son, hasta donde este acervo bibliográfico permite afirmar, una contribución metodológica de este proyecto más allá del propio sistema.

Hallazgos inesperados

Un indicador indirecto pero concreto de que la verificación de este proyecto fue real – no un ejercicio cosmético para llenar una guía – es la cantidad de defectos genuinamente inesperados que el propio proceso de desarrollo y auditoría encontró y corrigió, ninguno de ellos sembrado a propósito para “encontrar algo que reportar”. La comprobación del DSL de Structurizr detectó que `structurizr/cli` es, en el estado actual de ese ecosistema, una imagen retirada que imprime un aviso de desuso pero termina con código de salida 0 para cualquier entrada –válida o no–, por lo que una validación histórica contra esa imagen nunca habría sido una validación real (5). Un defecto en la generación de `credencial_qr_token` obligó a que Postgres generara el valor vía `DEFAULT` de columna en vez de que el backend lo insertara explícitamente (commit 1f1008e). Un `MailHealthIndicator` colgaba el `healthcheck` del contenedor cuando no había SMTP configurado, un defecto de configuración que solo se manifestaba en entornos sin correo real (commit 14992d6). Una `NoResourceFoundException` devolvía 500 genérico en vez de 404 – el mismo hallazgo que cerró la verificación de A05 en el 7 (commit 951fae5). Y una migración de Flyway (`V3__multas_multiples_por_prestamo.sql`, entre otras) fallaba en un despliegue limpio (`docker compose down -v` + reconstrucción completa) porque la fuente de migraciones estaba duplicada entre `database/migrations/` y `src/main/resources/db/migration/`, y `baseline-version` no reflejaba la versión más alta ya aplicada al esquema (commit 03821d7). Ninguno de estos 5 hallazgos estaba en el plan de trabajo original – todos surgieron de ejecutar el sistema de verdad, contra Docker real, en vez de razonar sobre el código sin correrlo. Esa es, en sí misma, la evidencia más directa de que el programa de calidad de este proyecto (4) no fue cosmético.

Implicaciones prácticas

Para una biblioteca universitaria real que evaluara adoptar un sistema como SGB-SaaS, la lectura honesta de este capítulo es la siguiente: el sistema cumple sus umbrales de rendimiento y cobertura con margen real, y su superficie de seguridad auditada (6 de los controles OWASP más relevantes para una aplicación con autenticación y datos personales) no llegó limpia a la primera revisión – tuvo defectos reales que se corrigieron, lo cual es una señal de proceso saludable, no una alarma, si se compara contra la alternativa de no auditar en absoluto. Su última pieza pendiente, la verificación de TLS end-to-end en un entorno de despliegue público real (A02), también se completó – el header `Strict-Transport-Security` se confirmó activo en producción el 18-ago-2026, dejando los 6 controles auditados con resultado cerrado. Lo que todavía no está listo para una decisión de adopción informada es la evidencia de usabilidad percibida por usuarios reales (RQ3, pendiente, $N = 0$) – la única pieza que sigue dependiendo de participantes externos al propio código, no de trabajo de ingeniería adicional dentro del repositorio. Una institución que considere este sistema debería tratar la Entrega Final

como una base arquitectónica y de calidad de código sólida – incluyendo su superficie de seguridad, ya verificada en su totalidad –, pero condicionar cualquier decisión de producción a que la pieza de usabilidad (SUS) se complete primero.

Ingeniería de requisitos

Stakeholders

El proyecto identifica cinco grupos de interesados reales, no hipotéticos: los cuatro roles de usuario final del sistema – **Lector** (estudiante o miembro de la comunidad universitaria), **Bibliotecario** (personal de mostrador, bajo conocimiento técnico esperado), **Gerente** (responsable de la biblioteca, además de las funciones de Bibliotecario puede anular multas y ver reportes) y **Administrador** (administrador técnico/institucional del sistema) – más el **docente-director** (Dr. Gleiston Cicerón Guerrero Ulloa, Ph.D.) como evaluador académico del PFC y fuente directa de retroalimentación formal (ver las observaciones de Entrega 1A, Entrega 1B y Tercera Entrega en docs/observaciones/OBSERVACIONES.md). El propio equipo de desarrollo (3 integrantes) actúa simultáneamente como responsable de elicitación, implementación y verificación – una restricción real de recursos declarada explícitamente en docs/requisitos/SRS-v1.0.0.md (sección 2.4) que explica, entre otras cosas, por qué ciertos requisitos quedan con evidencia parcial en vez de simularse como completos.

Técnicas de elicitación

El proyecto combina tres fuentes de requisitos, ninguna ficticia:

- **Historias de usuario y casos de uso**, en formato Connextra (“Como *rol*, quiero *acción*, para *beneficio*”) y Cockburn respectivamente, versionados como un archivo por HU/CU en docs/requisitos/historias/ y docs/requisitos/casos-de-uso/ para el módulo original del equipo, y consolidados en dos archivos únicos (historias-usuario.md, casos-de-uso.md) para las 5 HU/CU del módulo de Préstamos/Reservas/Multas – una inconsistencia real de convención entre módulos, documentada explícitamente en el SRS en vez de normalizada en silencio.
- **Hallazgos de auditoría de seguridad como fuente directa de requisitos no funcionales**: REQ-NF-006 (*rate limiting* de login) y REQ-NF-007 (auditoría de eventos de autenticación) no eran requisitos preexistentes – nacieron de un hallazgo real de la auditoría OWASP A07/A09 (ausencia total de límite de intentos y de registro de eventos), corregido dentro del mismo ciclo de desarrollo.
- **Inspección directa del código como último recurso declarado**, únicamente para los módulos construidos después de la Tercera Entrega que no llegaron a tener HU/CU dedicada – nunca presentada como si una HU real existiera. De los 46 requisitos, 10 (REQ-F-010, REQ-F-020 a REQ-F-022, REQ-F-025 a REQ-F-028, REQ-F-030, REQ-F-031) no citan ninguna HU/CU en la matriz de trazabilidad – los últimos 2 se agregaron al cierre de esta entrega, verificados directamente contra `matriz.csv` –, y otros 5 (REQ-F-017, REQ-F-018, REQ-F-019 parcialmente, REQ-F-023, REQ-F-024) citan un identificador de HU/CU que **no corresponde a ningún archivo real** del repositorio, verificado por búsqueda exhaustiva antes de escribir el SRS – en ambos casos, el rationale de esos requisitos en el SRS se declara explícitamente como inferido de la implementación, no como un hecho documentado previamente.

Los 46 requisitos: categorización MoSCoW

La fuente única de verdad es docs/trazabilidad/matriz.csv, validada automáticamente en cada ejecución de CI por scripts/validate-traceability.sh. Este capítulo resume su distribución agregada; el detalle de cada requisito individual (descripción, rationale, criterio de aceptación medible, método de verificación) vive en docs/requisitos/SRS-v1.0.0.md y en la propia matriz – no se reproducen aquí las 46 fichas completas.

Dimensión	Categoría	Cantidad (%)
Tipo	Funcional (REQ-F)	31 (67,4 %)
	No funcional (REQ-NF)	15 (32,6 %)
Prioridad MoSCoW	Must	23 (50,0 %)
	Should	23 (50,0 %)
	Could	0 (0,0 %)
	Won't	0 (0,0 %)
	verificado	23 (50,0 %)
Estado (matriz)	implementado	23 (50,0 %)
Total		46 (100,0 %)

Ningún requisito usa las categorías *Could* o *Won't* de MoSCoW – dato real, no una omisión de esta tabla: los 46 requisitos del sistema se consideraron, al momento de incorporarse, o bien imprescindibles (*Must*) o bien deseables dentro del alcance actual (*Should*); ninguno se registró formalmente como descartado o como candidato de prioridad baja.

Proceso de validación: 100 % de los requisitos *Must* verificados

Al cierre de este capítulo, los 23 requisitos de prioridad *Must* tienen estado *verificado* en la matriz – es decir, cuentan con prueba automatizada real y/o evidencia empírica directa contra el stack en ejecución, no solo con código que compila. Este 100 % es reciente: los dos últimos requisitos *Must* en estado *implementado* (REQ-F-001, registro de usuario, y REQ-NF-013, prevención de inyección SQL) carecían de prueba automatizada de regresión para parte de su criterio de aceptación – REQ-F-001 solo tenía cubierto el criterio de rechazo por correo duplicado, no el flujo exitoso ni el rechazo por contraseña corta; REQ-NF-013 solo tenía verificación manual puntual de la auditoría original, sin test de regresión permanente. Se cerraron agregando 2 tests nuevos (`AuthControllerTest.registro_passwordCorta_devuelve400ProblemDetail` y `AuthServiceTest.registroConPayloadDeInyeccionSeGuardaComoTextoLiteralSinLanzarExcepcion`, este último una regresión permanente del payload de inyección SQL verificado manualmente en la auditoría OWASP A03 original) en el commit `e1f0c25`.

Estabilidad del conjunto de requisitos: CHANGELOG-REQ.md

Nota de alcance (cierre de la Entrega Final). SRS-v1.0.0.md y CHANGELOG-REQ.md no se actualizaron con REQ-F-030/REQ-F-031 (agregados directo en `matriz.csv` al cierre, sin tiempo de pasar por el flujo SRS→CHANGELOG-REQ); esta sección cita fielmente lo que esos dos archivos dicen hoy (43 requisitos, no 46) en vez de inflar su conteo sin que el archivo fuente lo respalde – corregir ambos archivos queda fuera de alcance de esta tarea.

`docs/requisitos/CHANGELOG-REQ.md` compara el conjunto de requisitos entre la Tercera Entrega (`docs/requisitos/historico/SRS-v1.0.0.md`, 30 requisitos) y esta entrega (SRS-v1.0.0.md, 43 requisitos), comparando el **cuerpo completo** de cada requisito compartido, no solo su título – metodología que descartó explícitamente 2 falsos positivos (REQ-F-016, REQ-NF-009) que una primera versión del script de comparación marcó como “modificados” por un artefacto de extracción de texto (un separador Markdown capturado como parte del cuerpo), documentado en el propio CHANGELOG-REQ.md en vez de contar una modificación que en realidad no ocurrió. El resultado: 13 requisitos agregados, 2 genuinamente modificados (REQ-NF-012/TLS y REQ-NF-014/CSP, ambos reclasificados de “pendiente” a “parcialmente implementado” tras cerrar parte de su alcance), 0 eliminados – **tasa de estabilidad** = $1 - (2/43) = 0,9535 \approx 0,953$, es decir, **95,3 %**.

Nota de honestidad (hallazgo de este capítulo, no de CHANGELOG-REQ.md). Ese mismo archivo reporta también un “porcentaje verificado” de 21/43, es decir 48,8 %, calculado en el momento en que se escribió. Esa cifra quedó desactualizada dos veces desde entonces: primero por el cierre descrito en la sección anterior (commit `e1f0c25`, posterior a CHANGELOG-REQ.md), y después por el crecimiento de la matriz de 43 a 46 requisitos al cierre de esta entrega (REQ-F-030, REQ-F-031, ninguno de prioridad *Must*). El conteo real vigente de requisitos *verificado* es 23/46, es decir 50,0 %, no 48,8 % ni 53,5 %. Se señala aquí en vez de propagar silenciosamente una cifra desactualizada – CHANGELOG-REQ.md en sí no se corrige en esta tarea, por estar fuera de su alcance.

Checklist de calidad de requisitos (INCOSE v4)

Nota de alcance (cierre de la Entrega Final). `docs/checklists/incose2023-req.md` tampoco se actualizó con REQ-F-030/REQ-F-031 – evaluarlos exige releer el texto completo de cada uno contra las 15 características C1–C15, un trabajo analítico que no se puede improvisar de forma fiable bajo la presión de tiempo del cierre. Esta sección, igual que la anterior, cita fielmente los 43 requisitos que ese checklist evaluó de verdad, no 46.

`docs/checklists/incose2023-req.md` evalúa los 43 requisitos contra las 9 características individuales (C1–C9) del marco INCOSE v4 descrito en la 3.2 y las 6 características de conjunto (C10–C15), con el mismo criterio de honestidad metodológica del resto del proyecto: no se marca cumplimiento por defecto.

Resultado agregado, considerando solo las 8 características de contenido (C1–C8): 20 de 43 requisitos (46,5 %) pasan las 8 sin ninguna excepción; los 23 restantes (53,5 %) tienen al menos una excepción documentada con evidencia concreta – típicamente C5 (*Singular*: un requisito agrupa más de una acción o regla de negocio distinguible bajo un mismo id, 12 casos) o C2 (*Appropriate*: criterio de aceptación demasiado escueto frente a requisitos hermanos del mismo módulo, 5 casos); las de C8 (*Correct*) son las menos frecuentes (4 casos) pero las más específicas, incluyendo una que este mismo checklist encontró antes de que se cerrara: la afirmación de REQ-F-001/REQ-NF-013 de que ciertos criterios carecían de prueba automatizada, ya resuelta en el commit `e1f0c25` citado en la sección anterior.

Hallazgo sistémico de C9 (*Conforming*), declarado sin ambigüedad: ninguno de los 43 requisitos está redactado como una única cláusula “[condición] [sujeto] *shall* [acción] [objeto] [restricción]” del patrón formal de ISO/IEC/IEEE 29148:2018 (3.1). El formato real del SRS separa *Descripción* (sujeto + modal + acción + objeto, en prosa) de *Criterio de aceptación medible* (las condiciones/restricciones, en bullets aparte) – un híbrido Volere/Cockburn/Gherkin, no el patrón EARS de una sola oración. El propio checklist documenta esto como una desviación **estructural y sistemática**, no un defecto puntual de contenido de alguno de los 43 – razón por la cual, si se cuentan las 9 características en vez de solo las 8 de contenido, el resultado formal es 0 de 43 requisitos sin ninguna excepción (100 % con al menos una), enteramente explicado por este hallazgo de formato, no por 43 fallas de contenido independientes. No ocultar este resultado – aunque a primera vista parezca alarmante – es exactamente el criterio de honestidad metodológica que el propio checklist declara como su objetivo.

Sobre el conjunto (C10–C15): 4 de las 6 características de conjunto se evalúan como cumplidas con matices (*Complete, Feasible, Comprehensible, Able to be validated*), respaldadas por evidencia directa (los 46 requisitos vigentes hoy en la matriz, todos implementados y validados en CI vía `scripts/validate-traceability.sh` – esta es una comprobación mecánica de CI sobre el CSV completo, no una re-evaluación INCOSE, por eso sí se actualiza aquí a diferencia del resto de esta sección). Las otras 2 (*Consistent, Correct*) se marcan explícitamente con excepción: se identificaron 2 inconsistencias reales dentro del conjunto – la contradicción ya autodeclarada entre REQ-F-001/REQ-F-020 sobre el estado inicial de una cuenta tras el registro, y una nueva (encontrada por el propio checklist, no señalada antes) entre las dos cifras de latencia citadas por REQ-NF-003 para la misma condición de caché frío.

Trabajo futuro

Este capítulo enumera las líneas de trabajo que quedan explícitamente fuera del alcance de esta Entrega Final – no genéricas ni especulativas, sino los gaps reales ya identificados en el propio repositorio a lo largo de este documento. Cada una indica por qué quedó fuera de alcance y cómo se abordaría.

Completar la evidencia de usabilidad (SUS, $N \geq 15$) – PENDIENTE

El Bloque 5 de evaluación empírica sigue con $N = 0$ de los $N \geq 15$ participantes que exige la guía, bloqueado hasta contar con un despliegue público funcional donde participantes reales puedan interactuar con el sistema. Una corrida declarada como $N = 15$ se retiró por falta de trazabilidad a un export crudo del instrumento (ver OBS-08 en `docs/observaciones/OBSERVACIONES.md`, reabierta). Lo que sí queda implementado y verificado: el protocolo ya definido en su momento (plantilla de consentimiento informado, muestreo por conveniencia dentro de la comunidad UTEQ, anonimización por código de participante, `docs/etica/consentimientos/plantilla.md`), el instrumento SUS de 10 ítems (?) con sus 2 ítems suplementarios, y el pipeline de análisis (`scripts/sus-analysis.ipynb`) validado con datos mock. La corrida real – reclutamiento, sesiones de 15–20 minutos con registro de hora/duración, export crudo versionable sin PII, análisis y reporte – se difiere formalmente a la fase de despliegue en producción.

Análisis estático de SQL dinámico – CERRADO

El 6 señaló, al aclarar por qué `@Query(nativeQuery=true)` con parámetros bindeados no viola la prohibición de concatenación de cadenas de la guía, que un script de análisis estático dedicado para detectar SQL dinámico todavía no existía en el repositorio en ese momento. **Ese script ya existe y esta línea de trabajo queda cerrada:** `scripts/audit-sql-dynamic.sh` (añadido en el commit 8d86e2d, 2026-08-13) recorre `db/procs/*.sql` – no el código Java, sino el SQL fuente de los procedimientos, una capa que el análisis estático de SpotBugs/find-sec-bugs no cubre – buscando tres patrones prohibidos: la sintaxis literal `EXECUTE IMMEDIATE` (Oracle/T-SQL), la sintaxis literal `sp_executesql` (SQL Server), y cualquier sentencia `EXECUTE` a secas en PL/pgSQL (los 9 objetos de `db/procs/` son funciones con SQL fijo y parámetros nombrados bindeados; ninguno tiene un motivo legítimo para invocar `EXECUTE`). El script ignora líneas de comentario completo y ocurrencias de “execute” pegadas a un identificador o dentro de un literal de string, para no fabricar falsos positivos. Ejecutado de nuevo al escribir este capítulo (`bash scripts/audit-sql-dynamic.sh`): sale limpio, código 0, sobre los 11 archivos `.sql` actuales de `db/procs/`. La distinción que este mismo script debía respetar – entre concatenación real de variables Java dentro del literal de una consulta y `@Query` nativa con parámetros nombrados bindeados, ya verificada como segura en el 6 – sigue siendo el criterio de referencia si en el futuro se decide extender este análisis al código Java del backend, algo que hoy no es necesario porque el SQL dinámico prohibido nunca se construye ahí, sino potencialmente en el SQL de los procedimientos, que es justo lo que este script ya cubre.

Auditoría ZAP baseline scan – CERRADO (con un ítem real pendiente)

Una versión anterior de este capítulo declaraba la auditoría automatizada OWASP ZAP como aún no ejecutada. **Ya se ejecutó, dos veces, y queda documentada en el 7 (7.2):** un *baseline scan* (2026-08-16: 1 *High*, 1 *Medium*, 5 *Low*, 2 *Informational*) y un *Ajax Spider scan* más exhaustivo (2026-08-17: 2 *High*, 2 *Medium*, 5 *Low*, 2 *Informational*), ambos contra el stack Docker local, con artefactos JSON/HTML/MD versionados en `docs/mediciones/sec/zap/`. Los 2 hallazgos *High* del Ajax Spider se investigaron y explicaron con evidencia, no se ocultaron ni se retiraron del reporte: un *DOM-based XSS* (8 instancias) que `grep` confirma que no proviene de código propio (cero usos de `insertBefore`, `.innerHTML` o `bypassSecurityTrust*` en `frontend-angular/src/app/`) sino del *bundle* interno de Angular; y una *Vulnerable JS Library* (`ng-version 17.3.12`) que corresponde a un *build* escaneado **antes** del *upgrade* de Angular a 21.2.20 ya aplicado en `package.json`.

Único ítem genuinamente pendiente, ya declarado con honestidad en el propio 7: repetir la corrida de ZAP contra el *build* ya actualizado (Angular 21.2.20) para confirmar que el hallazgo de *Vulnerable JS Library* deja de aparecer – no hubo tiempo de hacerlo antes del cierre de esta entrega. Esa re-corrida, y solo esa, es la línea de trabajo futura real de este bloque; las 6 verificaciones manuales control-por-control (7.2) y las dos corridas automatizadas ya mencionadas no quedan pendientes.

Completar Lighthouse a 6 corridas (3 móvil + 3 escritorio) – CERRADO

Una versión anterior de este capítulo declaraba que solo existían 2 corridas totales, ambas de perfil móvil, frente a las 3 corridas por perfil (móvil + escritorio) que exige la guía. **Ese estado ya no es real:** el 7 (7.4) documenta hoy **12 corridas reales contra producción** – 2 rondas (17-ago y 18-ago) × 2 perfiles (móvil, escritorio) × 3 corridas –, con media y desviación típica reales por categoría y perfil, muy por encima del mínimo exigido. No fue solo completar una cuota: la primera ronda (17-ago) expuso un defecto real de *Cumulative Layout Shift* (CLS de 0,235–0,236 en escritorio, un pico de 0,338 en 1 de 3 corridas móviles), diagnosticado hasta su causa raíz en el código (`PortalPublicoComponent` sin espacio reservado para el grid del catálogo mientras carga) y corregido con un estado de carga tipo *skeleton* (commit `fced67d`), verificado primero en local (86 % de reducción de CLS, sin regresión de tests) y luego re-confirmado contra producción real en la segunda ronda (18-ago), comparando el hash del *bundle* servido antes de medir para no asumir el redeploy. No queda trabajo pendiente en este bloque.

Defensa CSRF dedicada para `/api/auth/refresh`

El ADR-012 (`docs/adr/adr-012-cookies-jwt.md`) documenta que la cookie del `refreshToken` pasó de `SameSite=Strict` a `SameSite=None` para que el navegador la enviara en peticiones cross-origin entre el frontend (`biblora-sgb.onrender.com`) y el backend en Render, requisito para que el *silent refresh* funcionara en producción. Verificado en el código actual (`SecurityConfig.java`, línea 55), la protección CSRF nativa de Spring Security está desactivada (`.csrf(csrf -> csrf.disable())`), y no existe en el repositorio ningún mecanismo de token CSRF ni validación manual de `Origin/Referer` (verificado por búsqueda directa, cero resultados). El único control activo es la lista blanca de CORS, que impide que un origen no autorizado *lea* la respuesta de `/api/auth/refresh`, pero no impide que dicho origen *envíe* la petición con la cookie adjunta – el navegador la adjunta igual por ser `SameSite=None`. Quedó fuera de alcance de esta entrega porque corregirlo exige un cambio coordinado backend/frontend (emitir el token en un lugar legible por JavaScript, p.ej. otra cookie no `HttpOnly` o un header de respuesta, y que `jwt.interceptor.ts` lo reenvíe en cada petición de refresh) que no se priorizó frente a los hallazgos de mayor severidad ya cerrados en esta entrega (ADR-012 original, ADR-003). Se documenta explícitamente como riesgo aceptado, no como algo ya mitigado por el diseño actual (ver ADR-012, sección *Consequences*). La línea de trabajo futura concreta es implementar un token CSRF de patrón *double-submit cookie*: al emitir el `refreshToken`, emitir también un valor aleatorio en una segunda cookie legible por JavaScript (`SameSite=None` pero sin `HttpOnly`); el frontend lo reenvía en un header custom (p.ej. `X-CSRF-Token`) en la petición de refresh, y el backend rechaza la petición si el valor del header no coincide con el de la cookie – un atacante cross-site puede lograr que el navegador adjunte la cookie, pero no puede leer su valor para construir el header a mano.

Salvaguarda contra auto-degradación del rol ADMIN

Al revisar el código real de `UsuarioAdminService.cambiarRol` (`backend-springboot/src/main/java/com/uteq/backend/servi` método `cambiarRol`, líneas 79–95) para este capítulo se confirmó un gap de seguridad real: el método reemplaza el conjunto de roles de *cualquier* `usuarioId` recibido por parámetro, sin comparar ese id contra el del ejecutor autenticado (*Authentication*) ni verificar si es el último usuario con rol ADMIN del sistema. En la práctica, un ADMIN autenticado puede degradarse a sí mismo – o degradar al único otro ADMIN existente – sin ninguna salvaguarda, quedando el sistema potencialmente sin ningún usuario con ese rol y sin una vía de recuperación dentro de la propia aplicación. Quedó fuera de alcance de corregirse en esta entrega porque se detectó durante la revisión de este mismo capítulo,

no antes – se documenta aquí en vez de corregirse apresuradamente sin las pruebas correspondientes, siguiendo el mismo criterio ya aplicado en otros hallazgos de este documento (declarar antes que parchear sin verificar). La línea de trabajo futura es agregar, en `cambiarRol`, una comprobación explícita de que (a) el usuario objetivo no sea el mismo ejecutor autenticado cuando el nuevo rol no incluya `ADMIN`, y (b) si lo es, que exista al menos otro usuario activo con rol `ADMIN` antes de aplicar el cambio – junto con un test de seguridad dedicado (análogo a `UsuarioAdminServiceTest` ya existente) que reproduzca ambos casos y falle si la salvaguarda se elimina en el futuro.

Conclusiones

Respuesta condensada a las preguntas de investigación

Versión condensada de la discusión completa de la 8.1 para las cuatro preguntas planteadas en la 1.2.

RQ1 (rendimiento). El sistema cumple ambos umbrales de latencia exigidos con margen amplio (p95 19,49 ms caliente, 7,50 ms frío), aunque la comparación entre escenarios no aísla limpiamente el efecto del caché por una limitación metodológica ya declarada – el umbral se cumple, la atribución causal exacta queda matizada.

RQ2 (seguridad). De 6 controles OWASP auditados, 4 presentaron un hallazgo real corregido dentro del mismo ciclo (66,7 %), y 2 se verificaron sin hallazgo real de código (A03 desde la corrida original; A02 tras completarse la verificación de TLS/HSTS contra el despliegue público real, 18-ago-2026) – los 6 controles quedan con resultado cerrado; la auditoría encontró y forzó a corregir defectos concretos, no fue un ejercicio cosmético.

RQ3 (usabilidad). Sin respuesta todavía: $N = 0$ de los $N \geq 15$ participantes exigidos por la guía. Queda como trabajo en progreso, bloqueado en la cadena de dependencias correcta (despliegue público $\rightarrow$ reclutamiento $\rightarrow$ aplicación del instrumento SUS), no como una omisión.

RQ4 (arquitectura híbrida). La estrategia CRUD-ORM/SP no perjudicó la trazabilidad (48,8 % CRUD-ORM, 18,6 % SP, 4,7 % híbrido, distribución medida sobre los 43 requisitos originales de 5.6 – la matriz creció a 46 requisitos tras el cierre, ver 9; el desglose porcentual no se recalculó para los 3 nuevos por alcance de esta corrección) ni la cobertura de pruebas (82,97 % de líneas), pero tuvo un costo real de implementación: una migración forzada de `@Procedure` a `@Query` nativa por un defecto documentado de Hibernate con procedimientos multi-OUT.

Contribuciones – recapituladas con evidencia del capítulo de resultados

El 1 enumeró cinco contribuciones de este proyecto. Con la evidencia empírica del 7 ya reunida, se sostienen así:

- Un sistema web funcional con 46 requisitos trazados, de los cuales el 100 % de los 23 de prioridad **Must** cuenta con evidencia de verificación real (9) – sostenida, no solo declarada: la matriz de trazabilidad (5.6) asigna un mecanismo de acceso a datos concreto a cada uno de los 46.
- Evidencia empírica reproducible y versionada como código: 5 corridas de k6 con análisis estadístico formal (Wilcoxon, Cliff's delta), cobertura JaCoCo real (82,97 % de líneas, commit `0d1474d`), y auditoría de accesibilidad/rendimiento con Lighthouse – las tres con datos crudos versionados en `docs/mediciones/`, no capturas de pantalla editadas a mano (7).
- Un checklist de calidad de requisitos INCOSE v4 aplicado con honestidad metodológica completa, documentando explícitamente qué requisitos no cumplen cada característica y por qué (`docs/checklists/incose2023-req.md`).
- Documentación arquitectónica versionada como código fuente: el modelo C4 completo en Structurizr DSL y 13 Architecture Decision Records (5).
- Una revisión de trabajos relacionados con 30 referencias verificadas individualmente contra Crossref – proceso que detectó y corrigió 5 errores de atribución de fuente antes de publicarse (2) – y una brecha de literatura identificada y sostenida con los resultados reales de este documento (8.2).

Estado del proyecto al cierre de esta entrega

SGB-SaaS cierra esta Entrega Final como el software (portada de este documento; sin tag v1.0.0 todavía), producto de 8 ramas de módulos de dominio mergeadas a `main` durante el ciclo de esta entrega (configuración paramétrica, préstamos avanzado, notificaciones y verificación, catálogo avanzado, panel de administración, seguridad de transporte, asistente con IA generativa, e infraestructura de CI), sobre una base de 46 requisitos especificados y trazados, con el 100 % de los requisitos **Must** verificados (9) y 82,97 % de cobertura de línea en 247 pruebas automatizadas del backend (7.3). De los cinco bloques de evaluación empírica exigidos por la guía, cuatro están completos con umbral cumplido (rendimiento,

seguridad, cobertura, calidad web), y uno – usabilidad – no se ha ejecutado todavía (7.9). El 10 deja protocolizadas las cinco líneas concretas que cerrarían esos gaps, incluida una vulnerabilidad de auto-degradación del rol ADMIN detectada durante la propia redacción de este documento. Esta distribución – logros reales junto a gaps declarados sin ocultarlos – es, en última instancia, la conclusión más importante de este documento: no que SGB-SaaS esté completamente terminado, sino que su estado real, con evidencia verificable en cada afirmación, es conocido con precisión al cierre de esta entrega.

Amenazas a la validez

Este capítulo analiza, en las 4 categorías estándar de amenazas a la validez de un estudio empírico de ingeniería de software, las limitaciones **reales** ya documentadas en el repositorio – no una lista genérica de amenazas de relleno que no correspondan a algo efectivamente observado o declarado durante este proyecto.

Validez de constructo

La amenaza de constructo más concreta de este proyecto está en el propio diseño del experimento de rendimiento (6.4, `docs/mediciones/perf/REPORT.md`): el escenario `cache_frio` se diseñó para medir el costo de una consulta al catálogo *sin* caché (el constructo de interés, para contrastarlo contra `cache_caliente`), pero su implementación real – un PRNG que elige una página aleatoria entre 0 y 999 en cada petición – hace que la gran mayoría de esas páginas queden fuera de rango de la tabla `libro` (poblada con datos de ejemplo de desarrollo, no con miles de registros), de modo que Spring Data devuelve una `Page` vacía sin necesidad de traer filas reales. En la práctica, `cache_frio` mide sobre todo el costo de una consulta `COUNT + SELECT` con resultado vacío, **no** el costo real de traer y serializar una página completa de libros sin caché – una construcción operacional distinta de la que el nombre del escenario sugiere. La comparación “caché caliente vs. caché frío” de este experimento, por tanto, **no aísla limpiamente el efecto del caché**: parte de la diferencia de latencia observada entre ambos escenarios refleja también el costo distinto de las consultas que cada uno ejecuta, no únicamente la presencia o ausencia de caché. Esta limitación está documentada explícitamente en el propio `REPORT.md` (punto 3 de su “Análisis breve”), no descubierta recién en este capítulo – se reproduce aquí porque es exactamente el tipo de amenaza de constructo que este capítulo debe declarar, en vez de reemplazarla por una amenaza genérica que no corresponda al proyecto real.

Validez interna

La corrida 1 de las 5 corridas de k6 (5, `docs/mediciones/perf/REPORT.md`) presenta una cola de latencia anómala en `cache_caliente` (p95=107,31 ms, frente a un rango de 6,25–12,36 ms en las 4 corridas siguientes) que no se repite en ninguna corrida posterior. La explicación documentada es *warm-up* de JVM/JIT y del *pool* de conexiones de HikariCP: la corrida 1 fue la primera ejecución de carga real contra un contenedor `sgb_backend` que llevaba varias horas sin tráfico significativo. Esto constituye una variable extraña (*confound*) no controlada directamente por el diseño experimental – no se descartó la corrida ni se repitió hasta obtener un resultado más limpio (lo que habría sido, en sí mismo, una amenaza distinta: sesgo de selección de datos), sino que se documentó y se incluyó en el agregado de las 5 corridas, confirmando además mediante el test estadístico pareado que el patrón `caliente > frio` se sostiene igual si se considera solo el subconjunto de las corridas 2–5. Una amenaza de validez interna distinta, propia de la auditoría de seguridad (5), es que el mismo equipo que implementó el sistema es quien ejecutó y documentó la auditoría OWASP: no existe una segunda parte independiente que haya intentado explotar el sistema sin conocimiento previo de su código fuente, lo que puede subestimar la superficie real de vulnerabilidades frente a una prueba de penetración por un equipo externo. En la misma línea de instrumentación, la suite frontend presentó 4 tests flaky dependientes de la hora local ($\geq 18:00$, diálogo de hora límite de retiro sin reloj inyectable), resueltos con funciones puras y `ClockService` y registrados como OBS-20 en `docs/observaciones/OBSERVACIONES.md`.

Validez externa

La amenaza de validez externa más relevante en este bloque es la ausencia de evidencia de usabilidad (System Usability Scale, SUS): la guía de esta entrega exige un mínimo de $N \geq 15$ participantes y el bloque sigue en $N = 0$ (7.5; corrida anterior retirada por falta de trazabilidad, ver OBS-08 en `docs/observaciones/OBSERVACIONES.md`, reabierta). Sin muestra no hay conclusión de usabilidad que generalizar – y esa es precisamente la amenaza declarada: RQ3 queda sin respuesta hasta la fase de despliegue. Para la corrida futura, el diseño ya anticipa sus límites: la muestra se reclutará **por conveniencia dentro de la comunidad UTEQ**, no de forma aleatoria ni fuera de ese contexto institucional. ? documentan, en su revisión crítica de prácticas de muestreo en investigación de ingeniería de software, que muestras pequeñas y no aleatorizadas – exactamente el perfil previsto de este estudio SUS – limitan severamente la generalización

de cualquier conclusión más allá de la muestra concreta estudiada, y recomiendan reportar explícitamente el método de reclutamiento y sus límites en vez de presentar el resultado como representativo de una población más amplia sin justificarlo. El protocolo completo – plantilla de consentimiento informado, tarea de *onboarding* de dos pasos, código de participante anónimo – ya está definido y se aplicará según lo definido cuando se ejecute la corrida (10.1).

Una segunda amenaza de validez externa, más estructural: todos los hallazgos de este proyecto provienen de un único contexto (una biblioteca universitaria ecuatoriana de bajo volumen, con un equipo de 3 personas sin dedicación exclusiva) – ninguna conclusión aquí debería generalizarse sin más a bibliotecas de otro tipo (públicas de gran escala, especializadas, con perfiles de carga o de usuario distintos).

Una tercera amenaza de validez externa, distinta de las dos anteriores, es el alcance del propio acervo bibliográfico reunido para la revisión de trabajos relacionados (2): de las 33 referencias que lo componen – cada una verificada individualmente contra el registro oficial de Crossref (autor, DOI y venue reales; ese mismo proceso de verificación detectó y corrigió 5 errores de atribución de fuente durante su construcción, evidencia del rigor con que se aplicó) – solo **12 califican como de “alto impacto”** en el sentido estricto que exige el criterio D6 de la guía (revista con cuartil JCR Q1/Q2, o conferencia CORE A del tipo ICSE/FSE/ASE/MSR/EASE/ESEM), por debajo del umbral de 20 que exige el nivel “Excelente” de la rúbrica – aunque sí se cumple el umbral de 30 referencias totales. Esto no refleja un déficit de esfuerzo de búsqueda: el dominio concreto de este PFC (gestión bibliotecaria universitaria con IA aplicada) es un nicho de aplicación con un volumen de publicación considerablemente menor en los venues de primer nivel de Ingeniería de Software que otras áreas más centrales de la disciplina (pruebas de software, arquitectura, DevOps); la literatura realmente disponible sobre este dominio se concentra, en cambio, en revistas especializadas de ciencias de la información (típicamente Q2) y en conferencias regionales o temáticas – reales e indexadas, pero fuera del conjunto CORE A que domina la producción de Ingeniería de Software general. Frente a esta limitación real y ya verificada de forma exhaustiva, la decisión tomada fue reportar el número exacto obtenido (12 de 33) en vez de completar el conteo con citas de venues de calidad no verificable – el mismo criterio de verificación cruzada contra Crossref que corrigió los 5 errores de atribución mencionados arriba es, en sí mismo, la evidencia de que ese criterio se aplicó de forma consistente en todo el acervo, no solo donde convenía a un conteo más favorable.

Validez de conclusión

La comparación estadística pareada entre `cache_caliente` y `cache_frio` (Wilcoxon de rangos con signo sobre el p95 de cada una de las 5 corridas, `scripts/perf-analysis.py`) es el ejemplo más concreto de amenaza a la validez de conclusión de este proyecto: el estadístico exacto de dos colas con $n = 5$ pares tiene un valor mínimo alcanzable de exactamente $p = 0,0625$ cuando las 5 diferencias apuntan en la misma dirección sin excepción – que es precisamente lo que ocurre aquí (`cache_caliente > cache_frio` en las 5 corridas) –, por lo que el resultado **no alcanza el umbral convencional de significancia** ($p < 0,05$) pese a un tamaño de efecto grande y consistente (Cliff’s delta = $-0,68$). Tratar este resultado como “no hay diferencia” sería una conclusión estadísticamente incorrecta: el límite real es la potencia estadística del tamaño de muestra mínimo exigido por la guía ($n = 5$ corridas), no evidencia sustantiva en contra del efecto observado – más corridas subirían la potencia sin que se espere que cambie la dirección de la conclusión, ya consistente en las 5 disponibles. Esta interpretación se documenta de forma explícita en `docs/mediciones/perf/REPORT.md` en vez de reportar solo el p -valor sin su contexto de potencia, que por sí solo induciría a la conclusión errónea de ausencia de efecto. Una segunda amenaza de conclusión, ya señalada en la sección de validez de constructo de este mismo capítulo, es que ese mismo resultado ($p = 0,0625$, efecto grande) se interpreta sobre una comparación que no aísla limpiamente el constructo de interés (el efecto del caché en sí) – ambas amenazas se refuerzan mutuamente y deben leerse juntas, no de forma aislada, para no sobre-interpretar la magnitud exacta del efecto del caché reportada aquí.

Declaraciones

Este capítulo reúne las declaraciones formales que la guía exige antes de las referencias bibliográficas. Sigue el mismo criterio de honestidad metodológica que el resto del documento: los datos verificables en el propio repositorio se presentan como tales, y los que exigen una decisión que solo puede tomar el equipo (financiamiento, conflictos de interés, distribución exacta de contribuciones) se dejan marcados de forma explícita como pendientes, en vez de decidirse unilateralmente en este documento.

Disponibilidad de código

El código fuente completo de SGB-SaaS (backend Spring Boot, frontend Angular, scripts de medición, infraestructura Docker) está disponible públicamente bajo licencia MIT (`LICENSE`, raíz del repositorio) en:

<https://github.com/mloorm14/sgb-saas>

El software citable de esta entrega corresponde al commit etiquetado como **v1.0.0** (resoluble con `git rev-list -n 1 v1.0.0`; ver portada de este documento). Los metadatos de citación siguen el formato Citation File Format 1.2.0 (`CITATION.cff`, raíz del repositorio), con ORCID verificado de los 3 autores.

Disponibilidad de datos

Los datos empíricos crudos que sustentan el 7 (salidas de k6, reportes JaCoCo en XML/CSV/HTML, resultados de Lighthouse, evidencia de auditoría OWASP control por control) están versionados junto con el código fuente en `docs/mediciones/`, en el mismo repositorio citado en la 13.1. **LICENSE** cubre el repositorio completo (*"the Software"*, sin excluir explícitamente los datos de `docs/`), por lo que estos datos quedan bajo la misma licencia MIT que el software – el equipo no ha declarado hasta la fecha una licencia de datos separada (p. ej. CC-BY) para este material, algo que podría formalizarse en una futura revisión de **LICENSE** si el equipo lo considera necesario.

Este conjunto de datos empíricos **no cuenta con un DOI propio independiente** – el equipo decidió no crear un archivado separado en un repositorio de datos (Zenodo permite archivar datasets con su propio DOI, distinto del DOI del software), y en su lugar lo cubre bajo el mismo DOI del software citado en la portada de este documento (10.5281/zenodo.22636466), ya que ambos se versionan juntos en el mismo repositorio y bajo la misma licencia MIT (13.1).

Los formularios de consentimiento informado firmados por participantes reales de las pruebas de usabilidad (cuando se ejecuten, 10.1) están explícitamente excluidos de este repositorio y de cualquier archivado público, por la razón declarada en la 13.4.

Disponibilidad de materiales

Los instrumentos y materiales usados para producir la evidencia de este documento están versionados como parte del propio repositorio de código, no como capturas de pantalla ni archivos externos no trazables:

- **Colección de peticiones HTTP:** `docs/postman/coleccion.json`, con 66 peticiones reales que cubren los módulos de Auth, Libros, Préstamos, Reservaciones, Multas, Configuración, Credencial QR, Notificaciones, Usuarios (Admin), Auditoría y Chatbot, incluyendo casos de error (401/403/404/422/423/429). Nota de honestidad: `docs/observaciones/OBSERVACIONES.md` (OBS-03) registra 39 peticiones como la cifra vigente en el commit `c6b372c` (cierre de la Tercera Entrega); el conteo real verificado directamente sobre el archivo actual para este capítulo es 66 – la colección creció junto con los 8 módulos de dominio mergeados durante esta entrega, y esa cifra antigua no se actualizó en su momento en `OBSERVACIONES.md`.
- **Scripts de carga (k6):** `k6/`, con semilla fija documentada (4.7) para reproducibilidad.
- **Script de análisis estadístico:** `scripts/perf-analysis.py` (bootstrap de IC 95 % sobre p95, semilla fija `BOOTSTRAP_SEED = 42`).
- **Configuración de Lighthouse CI:** `frontend-angular/lighthouseci.js`, con el perfil móvil/Slow 4G ya parametrizado y comentado, listo para ejecutarse contra el stack Docker real.
- **Protocolo de usabilidad SUS:** plantilla de consentimiento informado en `docs/etica/consentimientos/plantilla.md` (10.1).

Declaración ética

El tratamiento de datos personales y el protocolo de consentimiento informado para las pruebas de usabilidad SUS están documentados en detalle en `docs/etica/ETHICS.md`, verificado directamente para este capítulo. En síntesis:

- Los datos de ejemplo del sistema (`db/seed.sql`) usan metadatos bibliográficos públicos de libros reales publicados (ISBN, título, autor, editorial); ninguna contraseña se almacena en texto plano (hash BCrypt, costo 12); se verificó por auditoría de código que ningún endpoint expone el campo `passwordHash`.
- Los participantes de las pruebas SUS (cuando se ejecuten, 10.1) firman un consentimiento informado bajo la plantilla ya versionada, que declara explícitamente el propósito de la prueba, los datos recogidos (sin datos identificables, solo un código de participante P01, P02, ...), el mecanismo de anonimización, y el derecho a retirarse en cualquier momento sin consecuencias.

- Los formularios de consentimiento ya firmados por participantes reales **no se suben a este repositorio público bajo ninguna forma** – se conservan, según el texto verificado de `ETHICS.md`, ”por el equipo en un medio separado (físico o digital con acceso restringido)”. Nota de precisión: este documento inicialmente iba a describir ese medio como ”Drive institucional con acceso restringido al docente”, pero `ETHICS.md` no especifica ese detalle – solo declara que el medio es separado del repositorio y de acceso restringido, sin nombrar la herramienta concreta. Se reporta aquí lo que el archivo realmente dice, no el detalle asumido inicialmente.

Contribuciones de autoría (taxonomía CRediT)

La 13.1 resume la distribución de roles de autoría por integrante, siguiendo la taxonomía CRediT.

Table 11: Distribución de roles CRediT del equipo SGB-SaaS.

Rol CRediT	Cajas	Loor	Panamá
Conceptualization	No aplica – sin financiamiento externo declarado		
Data curation			
Formal analysis			
Funding acquisition			
Investigation			
Methodology			
Project administration			
Resources			
Software			
Supervision		Ver nota debajo de la tabla	
Validation			
Visualization			
Writing – original draft			
Writing – review & editing			

Justificación de la propuesta, con evidencia concreta verificada para este capítulo:

- **Cajas:** autor del mayor volumen de commits del proyecto (194 de ~390 commits con identidad atribuible, contando las variantes de firma `Theirvin1/TheIrvin/Irvin`), concentrados en el backend Spring Boot – servicios, controladores, la migración forzada de `@Procedure` a `@Query` nativa por el defecto de Hibernate (8), y la mayoría de los 247 tests automatizados del backend. Autor de los 8 módulos de dominio mergeados en esta entrega. Asignado también a la auditoría ZAP baseline pendiente (10.3).
- **Loor:** autor de la mayoría de los commits de `docs/` (este informe, arquitectura C4, ADRs, bibliografía verificada contra Crossref), de los scripts de medición (`k6/`, `scripts/perf-analysis.py`) y del análisis estadístico del 7. 150 commits (variantes `Marlon Loor/mlloorm14/Loor Marlon`).
- **Panamá:** autor de los componentes de frontend de Préstamos, Reservaciones y Multas (`frontend-angular/`) y de varias historias de usuario en `docs/requisitos/`. 44 commits. Asignado a la ejecución de las pruebas SUS pendientes (OBS-08, 10.1).

Nota sobre ”Supervision”. La guía CRediT reserva este rol típicamente para quien dirige el trabajo sin ser autor operativo directo del código – en este PFC, ese rol lo ejerce el docente-director (Dr. Gleiston Cicerón Guerrero Ulloa, ver portada), no uno de los 3 integrantes-autores. Se deja fuera de la tabla de columnas por integrante para no asignarlo incorrectamente a ninguno de los 3, y se señala aquí como punto abierto: el equipo debe decidir si desea listar al docente-director bajo este rol en la versión final de esta declaración.

Conflictos de interés

Los autores declaran que no existe ningún conflicto de interés en relación con el desarrollo de este proyecto ni con los resultados reportados en este documento. SGB-SaaS es un Proyecto de Fin de Curso (PFC) de carácter estrictamente académico, desarrollado sin patrocinio externo ni relación comercial, laboral o financiera con terceros que pudiera influir en el diseño, la ejecución o la interpretación de la evidencia presentada.

Financiamiento

Este proyecto no contó con financiamiento externo de ningún tipo. No recibió patrocinio institucional más allá del marco académico regular de la Universidad Técnica Estatal de Quevedo (UTEQ) en el que se desarrolla, ni financiamiento de terceros, ni becas de investigación específicas para su realización.

Uso de inteligencia artificial generativa

Durante el desarrollo del proyecto se utilizaron puntualmente herramientas de asistencia basadas en inteligencia artificial generativa (Claude, de Anthropic), como apoyo en tareas específicas y acotadas: consulta de documentación técnica, redacción asistida de fragmentos de este informe, y ejecución de tareas mecánicas de bajo riesgo (formateo, verificación de sintaxis, ejecución de comandos ya definidos por el equipo).

Las decisiones de arquitectura, diseño, alcance, priorización de requisitos, y los juicios sobre qué evidencia empírica era suficiente o qué limitación declarar en cada capítulo, correspondieron en su totalidad al equipo humano, en particular a quien lideró técnicamente el proyecto. Todo cambio al código o al repositorio pasó por revisión y aprobación humana explícita antes de aplicarse: ningún commit se generó ni se subió de forma autónoma sin intervención humana en el punto de decisión.

Agradecimientos

El equipo agradece al docente-director, Dr. Gleiston Cicerón Guerrero Ulloa, Ph.D., por su guía y retroalimentación constante a lo largo de las cuatro entregas de este PFC, que contribuyeron directamente a la calidad técnica y metodológica del proyecto.

Anexos

Este capítulo reúne el material suplementario exigido por la guía (Bloque B.17): la cadena de búsqueda bibliográfica ya documentada en el 2, el protocolo del instrumento SUS (aún sin ejecutar), el placeholder del pipeline CI/CD pendiente de captura manual, el checklist completo de Ralph et al. (2021) verificado contra `docs/checklists/ralph2021-engineering-research.md`, y la matriz de trazabilidad de requisitos parseada desde `docs/trazabilidad/matriz.csv` (43 filas de datos + encabezado, 11 columnas, verificadas con un lector CSV antes de transcribirse).

Cadena de búsqueda bibliográfica

La cadena que sigue es la misma documentada en la 2.1 del 2 (“Cadena de búsqueda de referencia”: la usada en las búsquedas originales del equipo y replicada en la búsqueda dirigida de ese capítulo). **No se inventa aquí una cadena nueva**; se traslada literalmente desde ese archivo fuente.

("library management system" OR "biblioteca" OR "library automation")
AND ("QR code" OR RFID OR chatbot OR "recommendation system" OR
"generative AI" OR "stored procedure" OR "cache-aside" OR Redis OR ORM)
AND (performance OR empirical OR evaluation OR "case study")

Contexto verificado en el mismo capítulo. Fuentes del acervo: IEEE Xplore, ACM Digital Library, Scopus, ScienceDirect (Elsevier) y SpringerLink (curación previa), más Crossref e IEEE Xplore en la búsqueda dirigida; ventana temporal 2017–2026; criterios de inclusión y exclusión declarados en la 2.1. El flujo de selección adaptado de PRISMA 2020 aparece en la 2.1.

Protocolo SUS (System Usability Scale)

Este anexo documenta únicamente el **protocolo y el cuestionario** del instrumento System Usability Scale (?). **No se reportan resultados ni datos de participantes**: al momento de redactar este documento, $N = 0$ de los $N \geq 15$ participantes exigidos por la guía, verificado en la 7.5 del 7 (“Datos crudos. Ninguno todavía.”).

Protocolo de aplicación

1. **Consentimiento informado.** Antes de la sesión, el participante firma la plantilla versionada en `docs/etica/consentimientos/plantilla.md` (los formularios firmados **no** se versionan en el repositorio; ver `docs/etica/ETHICS.md`).
2. **Anonimización.** Cada participante recibe un código (P01, P02, ...); no se registra nombre ni correo en los datos de análisis.
3. **Muestreo.** Por conveniencia dentro de la comunidad UTEQ (roles de dominio lector/bibliotecario), según el protocolo ya definido en la 4.4.

4. **Tarea previa al cuestionario.** Onboarding de prueba descrito en la plantilla de consentimiento (iniciar sesión, crear cuenta, editar, dar de baja, cerrar sesión), sin ayuda del equipo salvo bloqueo total.
5. **Cuestionario.** Tras la tarea, se administran los 10 ítems de Brooke (1996) en español, con escala Likert de 1 a 5 (abajo). Duración estimada total de la sesión: 15–20 minutos (según la plantilla de consentimiento).
6. **Puntuación (cuando existan datos).** La fórmula estándar del instrumento produce una puntuación agregada 0–100 por participante. **Aquí no se aplica:** no hay respuestas que puntuar.

Cuestionario SUS – 10 ítems originales de Brooke (1996) en español

Instrucciones al participante: indique su grado de acuerdo con cada afirmación sobre el sistema que acaba de usar, marcando un solo valor en la escala de 1 a 5.

Escala Likert (idéntica en los 10 ítems)

1 = Totalmente en desacuerdo = En desacuerdo = Neutral = De acuerdo = Totalmente de acuerdo

1. Creo que me gustaría utilizar este sistema con frecuencia. (1) (2) (3) (4) (5)
2. Encontré el sistema innecesariamente complejo. (1) (2) (3) (4) (5)
3. Pensé que el sistema era fácil de usar. (1) (2) (3) (4) (5)
4. Creo que necesitaría el apoyo de una persona con conocimientos técnicos para poder usar este sistema. (1) (2) (3) (4) (5)
5. Encontré que las diversas funciones de este sistema estaban bien integradas. (1) (2) (3) (4) (5)
6. Pensé que había demasiada inconsistencia en este sistema. (1) (2) (3) (4) (5)
7. Imagino que la mayoría de la gente aprendería a utilizar este sistema muy rápidamente. (1) (2) (3) (4) (5)
8. Encontré el sistema muy engorroso de usar. (1) (2) (3) (4) (5)
9. Me sentí muy confiado/a usando el sistema. (1) (2) (3) (4) (5)
10. Necesité aprender muchas cosas antes de poder manejarme con este sistema. (1) (2) (3) (4) (5)

Los ítems impares son afirmaciones positivas y los pares negativas, según el diseño original del instrumento (?). La puntuación agregada, cuando existan respuestas reales, se calculará con la fórmula estándar de Brooke; **no se inventa aquí ninguna puntuación.**

Pipeline CI/CD

El flujo de integración continua del proyecto está definido en `.github/workflows/ci.yml` (disparado en `push/pull_request` hacia `main` y `develop`). La guía pide evidencia visual de **tres corridas verdes consecutivas**. Esas capturas **aún no están en el repositorio**: en su lugar se documenta aquí el pipeline real versionado, job por job, para que la evidencia visual futura sea verificable contra esta especificación (no se fabrican aquí capturas ni se declara un estado de CI no verificado en este anexo).

- **build-and-test** (`ubuntu-latest`, servicios Postgres + Redis): checkout, JDK 21, cliente `psql`, inicialización de esquema y procedimientos, auditoría de SQL dinámico en procedimientos, build + tests Maven, archivo de reporte SpotBugs + find-sec-bugs, validación de matriz de trazabilidad.
- **frontend** (`ubuntu-latest`): checkout, Node 20, instalación de dependencias, lint (pendiente), tests en modo headless, build de producción.

Checklist Ralph et al. (2021) — Engineering Research

El contenido de esta sección es la transcripción tabular del checklist real versionado en `docs/checklists/ralph2021-engineering-` (fecha 2026-08-13, commit base `6696bf1`, estándar “Engineering Research” de ?). **No se resume ni se alteran los veredictos**: se reformatea el mismo contenido a tablas `longtable`.

Leyenda. Cumple / Parcial / No cumple / N/A (no aplica al tipo de artefacto).

El checklist completo se distribuye en las tablas siguientes: atributos esenciales (14.1), atributos deseables (14.2), atributos extraordinarios (14.3), autoexamen de antipatronos (14.4) y el resumen de cumplimiento (14.5).

Atributos esenciales

#	Ítem
#	Ítem
E1	Describe el artefacto propuesto con detalle adecuado
E2	Justifica la necesidad, utilidad o relevancia del artefacto propuesto
E3	Evalúa conceptualmente el artefacto: discute fortalezas, debilidades y limitaciones
E4	Evalúa empíricamente el artefacto usando: <i>action research</i> , <i>case study</i> , experimento controlado, simulación cuantitativa, <i>benchmarking study</i> , u otro método
E5	Indica claramente cuál de esas metodologías empíricas se usó
E6	O BIEN discute alternativas del estado del arte (con sus fortalezas/debilidades/limitaciones) O BIEN explica por qué no existen O BIEN argumenta de forma convincente que no existen
E7	O BIEN compara empíricamente el artefacto con una o más alternativas del estado del arte O BIEN lo compara con <i>benchmarks</i> establecidos O BIEN justifica por qué no se comparó
E8	Los supuestos (si los hay) son explícitos, plausibles y no se contradicen entre sí ni con los objetivos de la contribución
E9	Usa notación de forma consistente (si se usa alguna notación)

Atributos deseables

Table 13: Checklist de atributos deseables

#	Ítem
#	Ítem
D1	Provee materiales suplementarios: código fuente (si el artefacto es software) y conjuntos de datos de entrada (si aplica)
D2	Justifica, con motivos prácticos o éticos, cualquier elemento faltante del paquete de replicación
D3	Discute la base teórica del artefacto
D4	Provee argumentos de corrección para las contribuciones analíticas y teóricas clave (p. ej. teoremas, análisis de complejidad, demostraciones matemáticas)
D5	Incluye uno o más ejemplos funcionando para ilustrar el artefacto
D6	Evalúa el artefacto en un contexto relevante para la industria (p. ej. proyectos de código abierto ampliamente usados, programadores profesionales)

Atributos extraordinarios

Table 14: Checklist Ralph et al. (2021) — atributos extraordinarios (aspiracionales; no reclamados)

#	Ítem	Estado	Nota
#	Ítem	Estado	Nota
X1	Contribuye a la comprensión colectiva de prácticas o principios de diseño	N/A	No se reclama — alcance de PFC académico, no de investigación
X2	Presenta innovaciones revolucionarias con beneficios obvios en el mundo real	N/A	No se reclama, por el mismo motivo.

Auto-verificación de antipatrones

Esta subsección corresponde al autoexamen del checklist (no forma parte del conteo de cumplimiento de los 17 ítems anteriores).

Table 15: Antipatrones del checklist de atributos esenciales
docs/checklists/ralph2021-engineering-research.md

Antipatrón	¿Se evitó?	Nota
Antipatrón	¿Se evitó?	Nota
Sobreestimar la novedad de la contribución	Evitado	Capacidad de innovación
Omitir detalles conceptuales clave centrándose solo en aspectos incidentales de implementación	Evitado	Capacidad de implementación
La evaluación consiste únicamente en recoger opiniones de usuarios	Evitado	Número de usuarios
La evaluación consiste únicamente en datos de rendimiento cuantitativos no comparados contra <i>benchmarks</i> o alternativas establecidas	Riesgo parcial	El benchmarking es una buena práctica
Diseño no experimental (un solo grupo, sin repetición)	Riesgo parcial	El diseño experimental es una buena práctica
Evaluación usando ejemplos de juguete (a veces presentados como “estudios de caso”)	Evitado	Los ejemplos de juguete no son adecuados para la evaluación

Resumen de cumplimiento (verificado)

Conteo recalculado al transcribir las 17 filas del checklist fuente (9 esenciales + 6 deseables + 2 extraordinarios):

Table 16: Resumen de cumplimiento Ralph et al. (2021), idéntico al del archivo fuente docs/checklists/ralph2021-engineering-research.md

Categoría	Cumple	Parcial	No cumple	N/A
Esenciales (9 ítems)	5	2	1	1
Deseables (6 ítems)	4	0	1	1
Extraordinarios (2 ítems)	0	0	0	2
Total (17 ítems)	9	2	2	4

Matriz de trazabilidad

La 14.6 transcribe **las 43 filas de datos** de docs/trazabilidad/matriz.csv, leídas con el módulo csv de Python (no copia visual). Verificación previa a la transcripción: 1 fila de encabezado + 43 filas de datos = 44 filas totales; 11 columnas en todas las filas (id_requisito, tipo, prioridad_moscow, historia_usuario, caso_de_uso, modulo_codigo, endpoint_api, prueba_automatizada, tipo_acceso, evidencia_empirica, estado). Las celdas con rayas largas (“—”) en el CSV se conservan como tales.

ID	Tipo	MoSCoW	HU	
ID	Tipo	MoSCoW	HU	
REQ-F-001	funcional	Must	HU-AUTH-01	
REQ-F-002	funcional	Must	HU-AUTH-02	
REQ-F-003	funcional	Must	HU-AUTH-03	
REQ-F-004	funcional	Must	HU-AUTH-04	
REQ-F-005	funcional	Must	HU-LIB-01	
REQ-F-006	funcional	Must	HU-LIB-02	
REQ-F-007	funcional	Must	HU-01 (Cajas)	
REQ-F-008	funcional	Must	HU-02 (Cajas)	
REQ-F-009	funcional	Must	HU-F02 (Panama)	
REQ-F-010	funcional	Should	—	
REQ-F-011	funcional	Must	HU-03 (Cajas); HU-F03 (Panama)	
REQ-F-012	funcional	Must	HU-F03 (Panama, inferida — el mismo componente cubre creación y listado, no hay HU dedicada solo a listar)	
REQ-F-013	funcional	Must	HU-F01 (Panama)	
REQ-F-014	funcional	Must	HU-04 (Cajas)	
REQ-F-015	funcional	Must	HU-05 (Cajas)	
REQ-F-016	funcional	Must	HU-F04 (Panama)	
REQ-NF-001	no funcional	Must	HU-AUTH-03	
REQ-NF-002	no funcional	Must	HU-AUTH-04	
REQ-NF-003	no funcional	Should	— (requisito a nivel de configuración, ver CU-LIB-01)	
REQ-NF-004	no funcional	Must	HU-01 (Cajas, lado SP); HU-AUTH-06 (Marlon, lado ORM)	
REQ-NF-005	no funcional	Should	N/A - decisión arquitectónica	
REQ-NF-006	no funcional	Must	HU-AUTH-05	
REQ-NF-007	no funcional	Must	HU-AUTH-06	
REQ-NF-008	no funcional	Should	N/A - decisión arquitectónica	
REQ-NF-009	no funcional	Should	N/A - decisión arquitectónica	
REQ-NF-010	no funcional	Must	HU-AUTH-07	
REQ-NF-011	no funcional	Must	HU-AUTH-07	
REQ-NF-012	no funcional	Should	N/A - decisión arquitectónica	
REQ-NF-013	no funcional	Must	HU-01 (Cajas); HU-LIB-02 (Marlon)	
REQ-NF-014	no funcional	Should	N/A - decisión arquitectónica	
REQ-NF-015	no funcional	Should	N/A - decisión arquitectónica	
REQ-F-017	funcional	Should	HU-CFG-01	
REQ-F-018	funcional	Should	HU-PRE-03	
REQ-F-019	funcional	Should	HU-PRE-04	
REQ-F-020	funcional	Should	—	
REQ-F-021	funcional	Should	—	
REQ-F-022	funcional	Should	—	
REQ-F-023	funcional	Should	HU-ADM-01	
REQ-F-024	funcional	Should	HU-AUD-01	
REQ-F-025	funcional	Should	—	
REQ-F-026	funcional	Should	—	
REQ-F-027	funcional	Should	—	
REQ-F-028	funcional	Should	—	

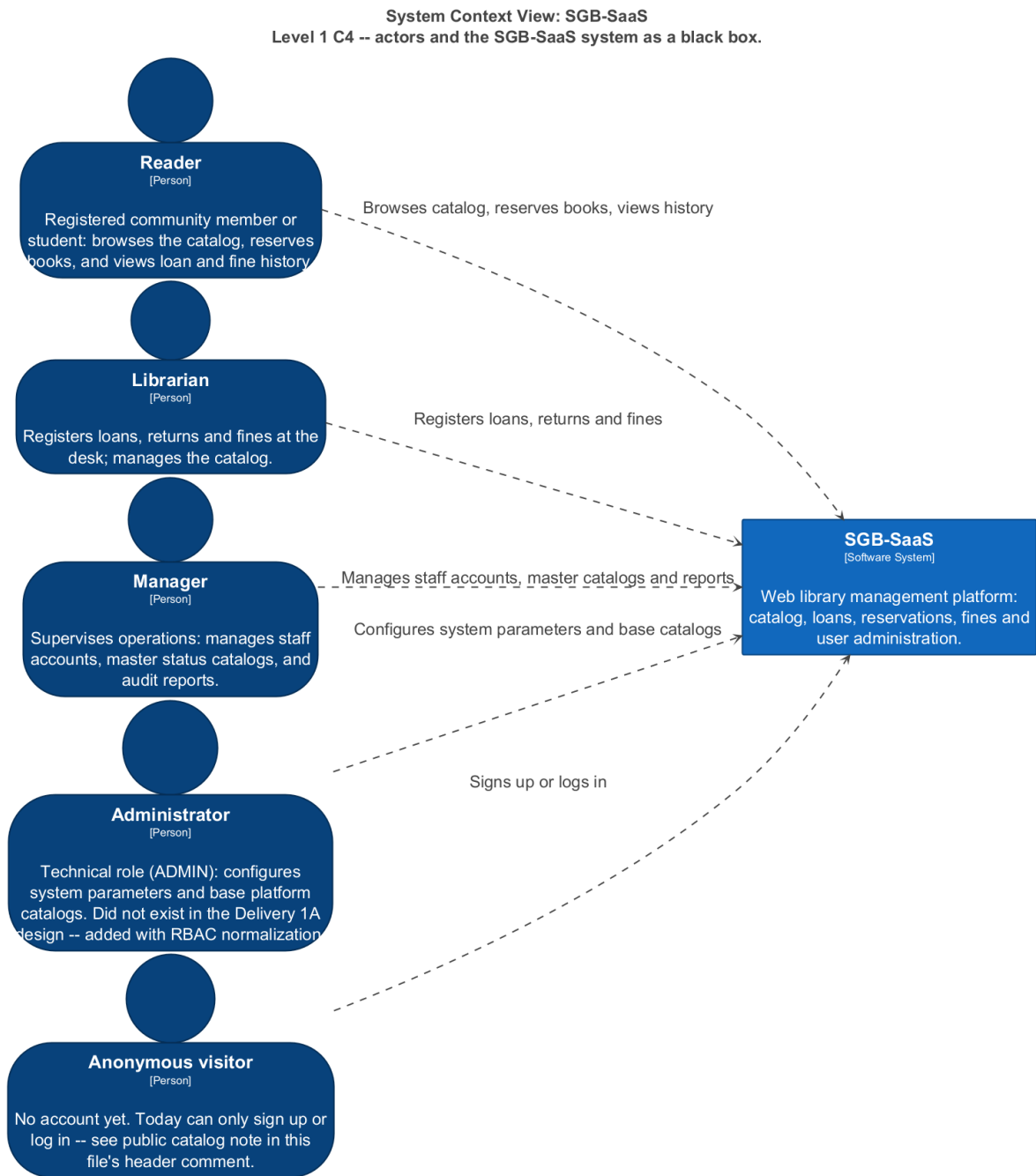


Figure 2: Modelo C4, Nivel 1 (Contexto): los cinco actores del sistema y sus interacciones con SGB-SaaS como caja negra. Exportado automáticamente desde `workspace.dsl`.



Figure 3: Modelo C4, Nivel 2 (Contenedores): Frontend Angular, Backend Spring Boot, PostgreSQL y Redis, alineados 1:1 con `docker-compose.yml`. Exportado automáticamente desde `workspace.dsl`.

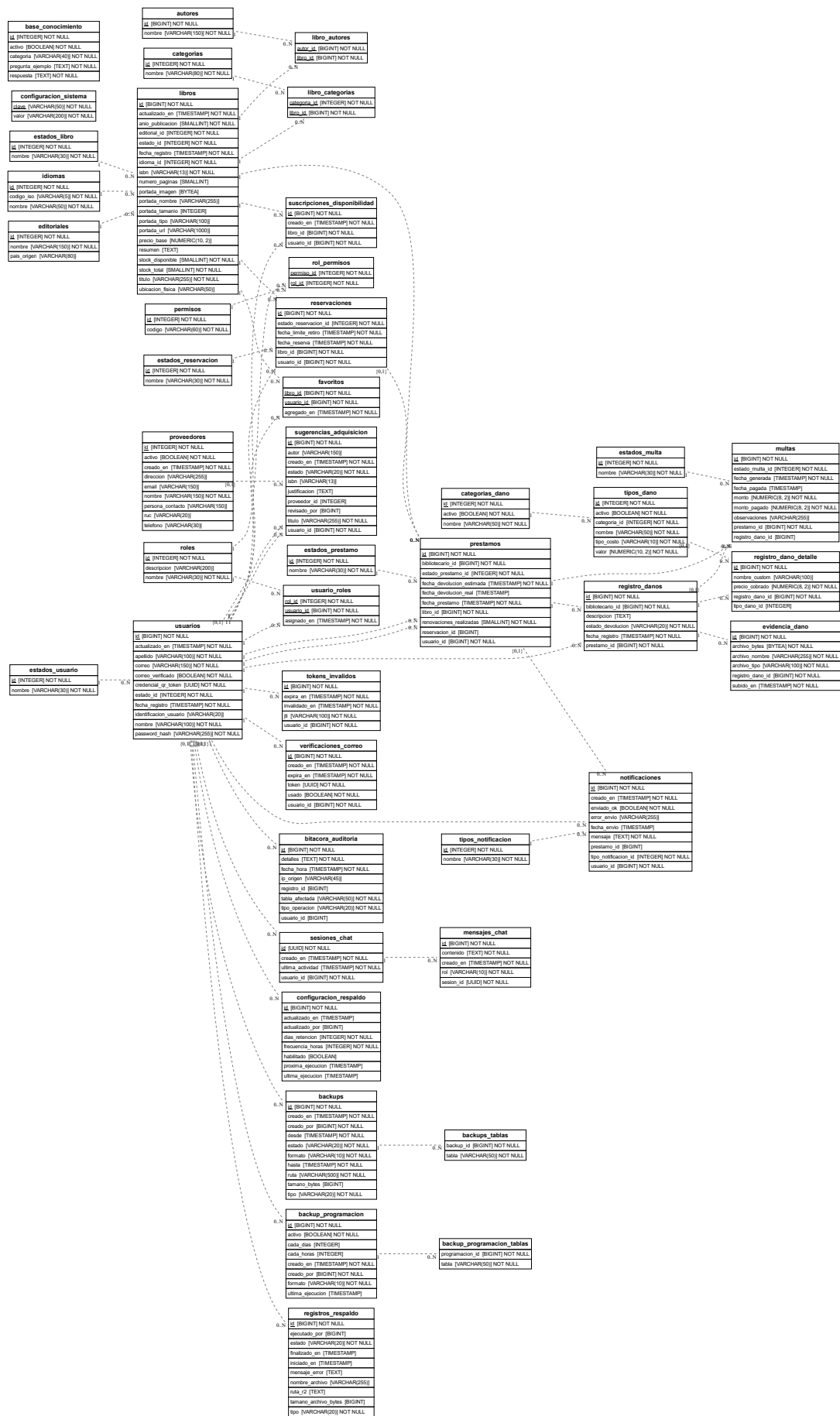


Figure 4: Diagrama entidad-relación real, generado automáticamente con **eralchemy2** a partir del esquema PostgreSQL reconstruido desde las 35 migraciones **43Flyway** de **database/migrations/**. Reemplaza a **docs/diagramas/Diagrama_ER.png**, que correspondía por error a un dominio clínico ajeno a este proyecto (ver OBS-21

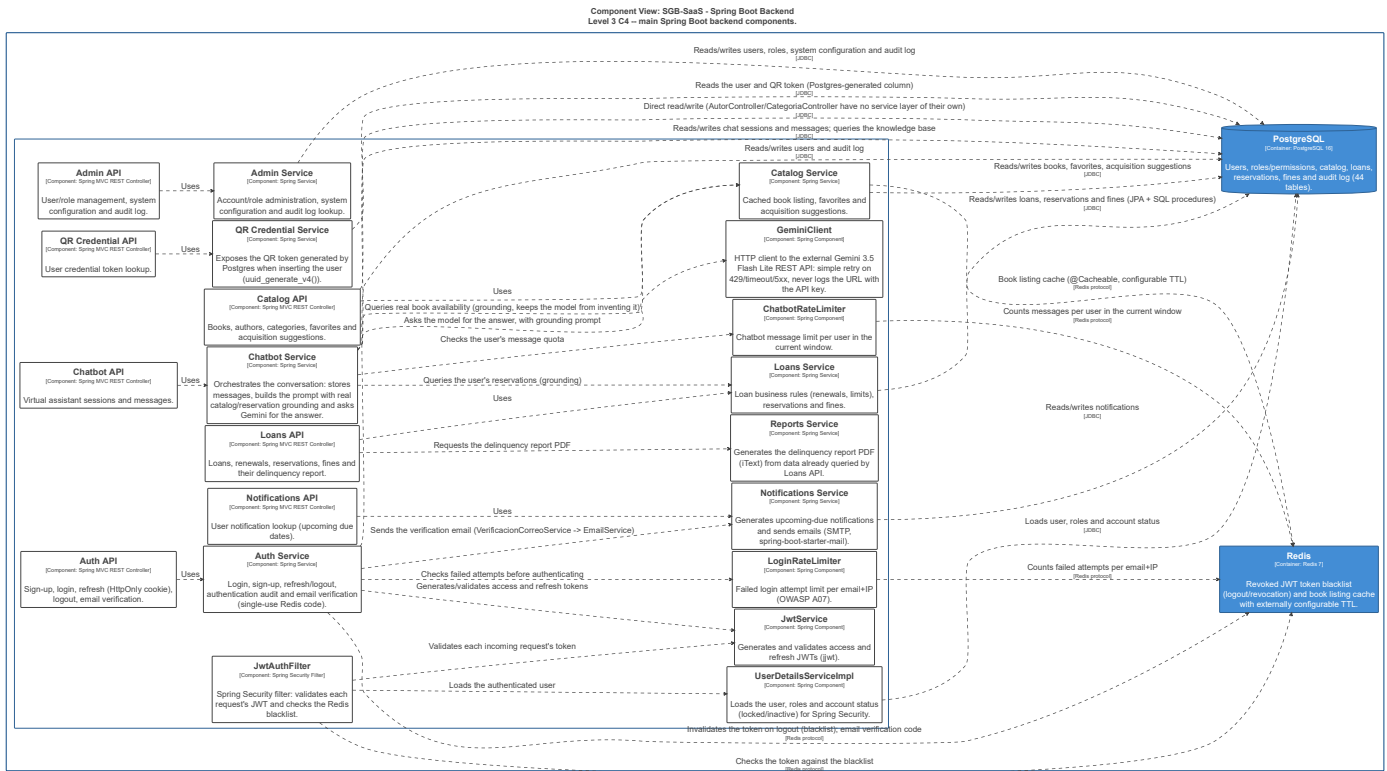


Figure 5: Modelo C4, Nivel 3 (Componentes) del backend Spring Boot – 21 componentes agrupados por dominio (capa API, capa de servicio, seguridad e integración externa), con sus relaciones verificadas contra el código real. Conversión a PDF del SVG fuente versionado en docs/arquitectura/c4-nivel3-componentes.svg (pdflatex no soporta SVG directamente).

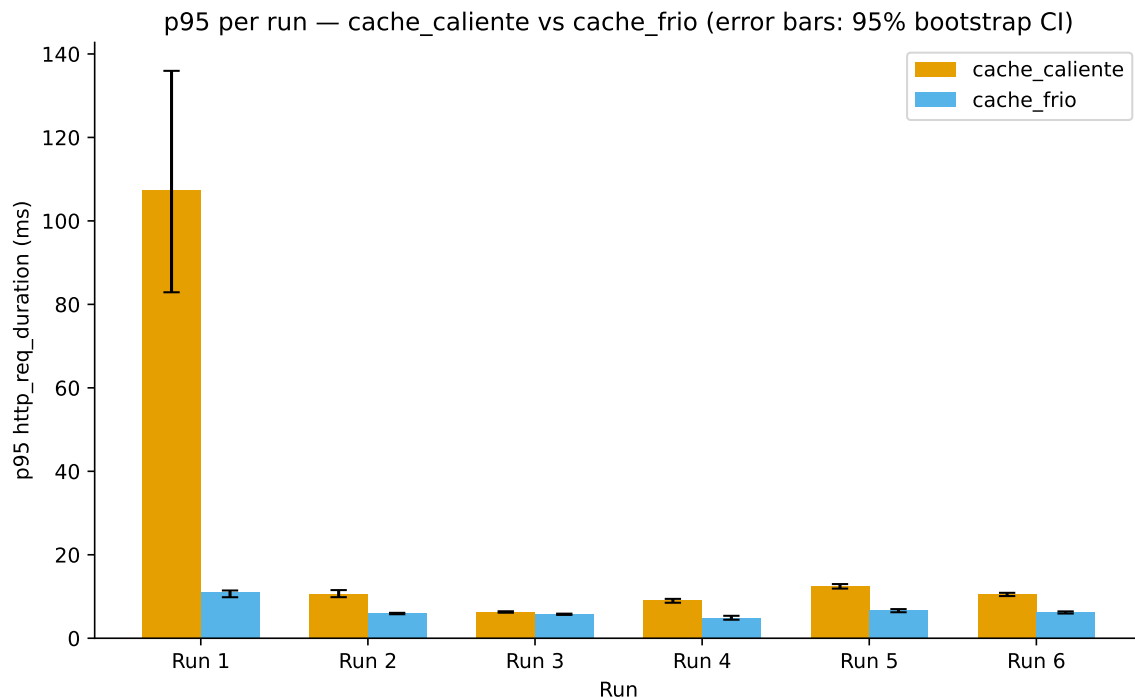


Figure 6: p95 de `http_req_duration` por corrida, `cache_caliente` vs. `cache_frio`, con barras de error de IC 95% (bootstrap, 2000 réplicas, semilla 42). Paleta accesible a daltonismo (Okabe-Ito).

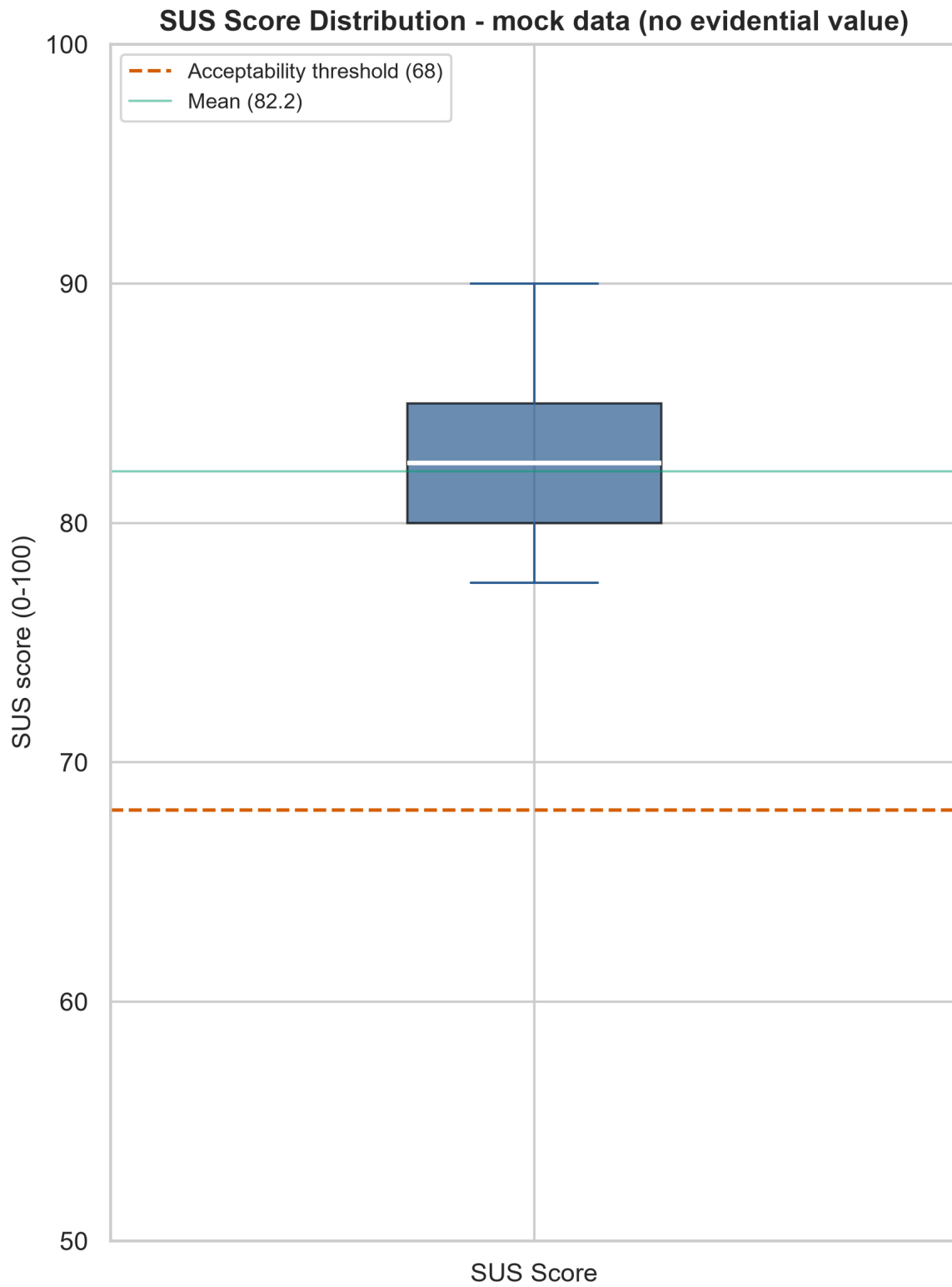


Figure 7: Distribución de puntuaciones SUS con datos mock/sintéticos de prueba del pipeline (sin valor evidencial, $N = 0$): la muestra real de usabilidad queda reservada para fases futuras con despliegue en producción.

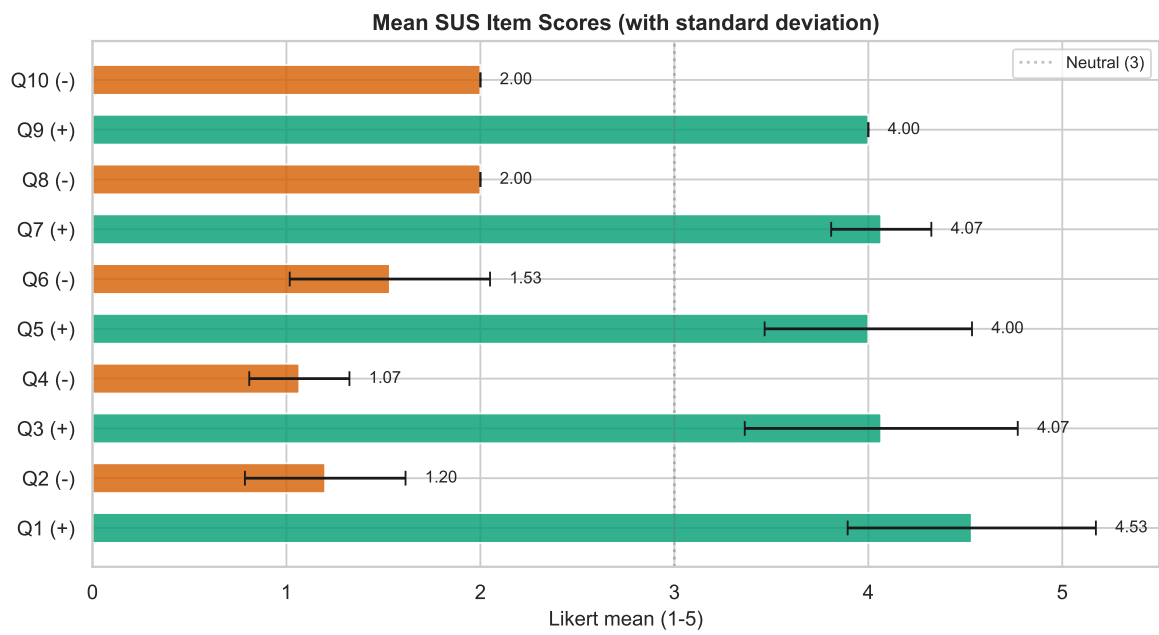


Figure 8: Desglose por ítem Q1–Q10 del instrumento SUS con datos mock/sintéticos de prueba del pipeline (sin valor evidencial, $N = 0$): muestra real reservada para fases futuras.